

QA Portal

Architecture, Data Model & Implementation Reference

System	QualityOps QA Portal
Scope	Frontend, API, database, storage, cache, security and workflows
Source baseline	Repository implementation as of 23 August 2026
Audience	Engineering, QA governance, DBA, operations and auditors

Document purpose

This document explains how the portal is assembled, how data moves through it, which database tables own each business capability, and which implementation constraints are significant for support, change control and production operation.

Important: This is a code-derived architecture baseline. Physical Oracle column types, constraints and execution plans should be confirmed against the deployed schema before production DDL or capacity decisions.

1. Executive architecture summary

QualityOps is a modular monolithic QA governance and test-management platform. The browser client is a single React application; FastAPI exposes domain-oriented REST routes; Oracle is the source of truth; Redis is the shared cache and startup-lock coordinator in the current Docker Compose deployment; uploaded evidence is stored on a persistent Docker volume or configured bind mount. The current implementation contains 50 mapped tables and 389 API route registrations across 22 router modules.

- Primary business chain: QA intake → linked delivery workflow(s) → test project/repository → execution → defects → sign-off and reporting.
- Authorization combines JWT authentication, role checks, department scope, record ownership, assignment and workflow status.
- Business traceability is stored in ApprovalAction; platform/security activity is stored separately in AuditLog.
- Testcase identity is separated from version snapshots. TestCase is the stable key; TestCaseVersion and TestCaseVersionStep preserve revision history.
- The runtime is horizontally scalable at the API layer, but Oracle and the upload volume remain shared state; Redis coordinates cache consistency and selected startup work.

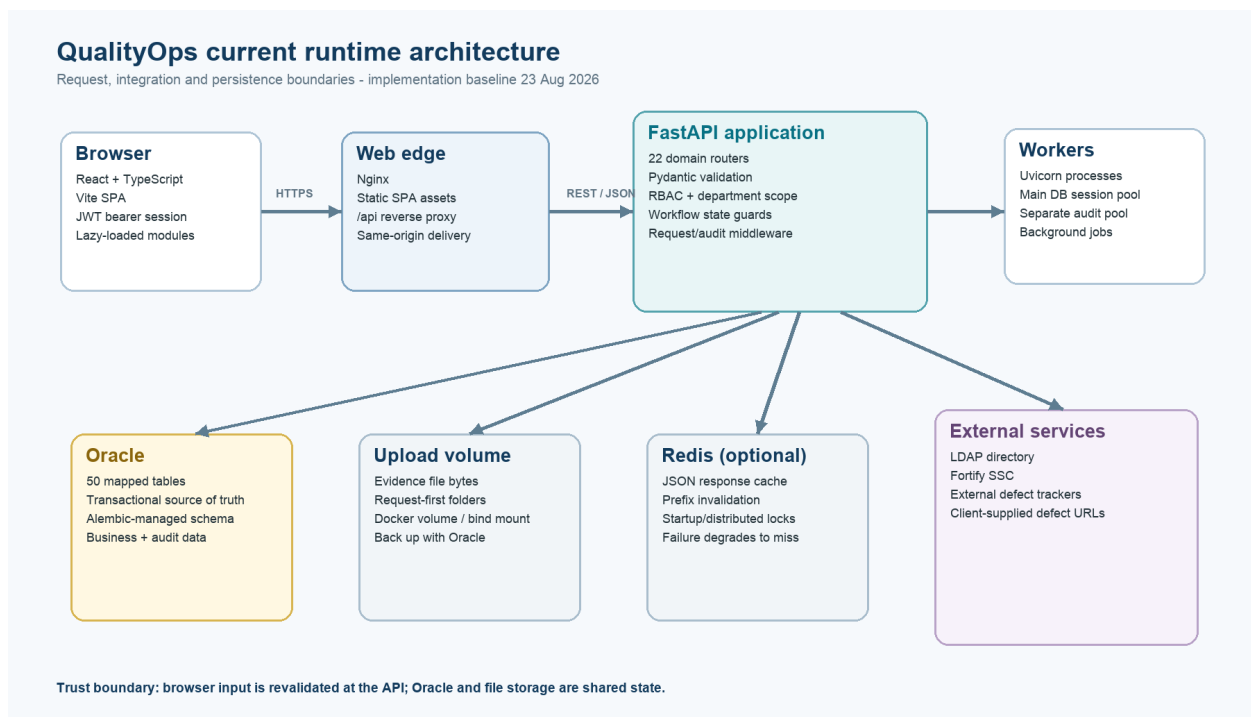


Figure 1. Current runtime architecture, integration boundaries and shared-state dependencies.

2. Technology and deployment stack

Layer	Implementation	Responsibility	Significant behavior
Browser UI	React, TypeScript, Vite, React Router	Navigation, forms, dashboards, client validation	Lazy-loaded domain chunks; one protected layout persists across route changes.
Web edge	Nginx container	Serves built SPA; proxies /api	Frontend and API remain same-origin in Docker deployment.
Application API	FastAPI, Pydantic, SQLAlchemy	Business validation, workflow state, serialization	Multiple worker processes; synchronous DB sessions per request.
Primary data	Oracle via python-oracledb thin mode	Transactional system of record	Application refuses non-Oracle URLs; NUMBER(1) boolean semantics matter.
Cache	Redis 7	Dashboard/reference caching and locks	Best-effort; requests continue when unavailable.
Files	Persistent upload volume	Evidence, attachments, comment images	DB contains metadata/path; backup must include both DB and volume.
Identity	Local users + optional corporate LDAP	Credential validation and JIT provisioning	JWT bearer token carries identity; DB provides current role/department state.

3. Component architecture and data flow

3.1 Frontend boundaries

App.tsx defines one authenticated shell and lazy-loads Functional, SAST, DAST, Suppression, Performance, Governance and Test Management modules. Shared components provide tables, badges, evidence, activity, identity signing and workflow decision panels. AuthContext owns session state; api.ts centralizes HTTP behavior.

3.2 API boundaries

Router / prefix	Primary responsibility	Principal tables
/api/auth	Login, LDAP JIT provisioning, users, roles	qap_users, qap_user_roles, qap_user_departments
/api/qa-requests	Gateway intake and linked workflow creation	qap_requests, qap_request_documents
/api/functional-requests	Functional lifecycle and tester assignment	qap_functional_requests, qap_readiness_checklist_items
/api/sast, /api/dast	Security workflows, targets/components/findings	qap_sast_*, qap_dast_*
/api/performance-requests	Performance lifecycle and tester assignment	qap_performance_requests, qap_performance_checklist_items

Router / prefix	Primary responsibility	Principal tables
/api/suppressions	False-positive governance	qap_suppression_requests, qap_suppression_items
/api/test-projects	Project lifecycle and access	qap_test_projects, members, grants
/api/test-repository	Folders, cases, versions and approval	qap_test_folders, qap_test_cases, qap_test_case_versions
/api/test-execution	Cycles, assignments, runs and links	qap_test_cycles, qap_test_executions, qap_test_execution_runs
/api/defects	Defect workflow and traceability	qap_defects, qap_defect_case_links
/api/dashboard, reports	Cross-domain read models	Queries multiple domain tables; Redis cache
/api/approvals, /api/audit	Business history and request audit	qap_approval_actions, qap_audit_logs

3.3 Request transaction sequence

1. React sends bearer-authenticated REST request.
2. FastAPI middleware assigns/carries X-Request-ID and starts timing.
3. Dependency resolves JWT identity and eagerly loads role/department memberships.
4. Router enforces role, department, ownership, assignment and valid status transition.
5. SQLAlchemy writes domain records plus ApprovalAction in a transaction.
6. Response returns; middleware invalidates affected dashboard cache prefixes.
7. Background audit task writes AuditLog using the separate audit connection pool.

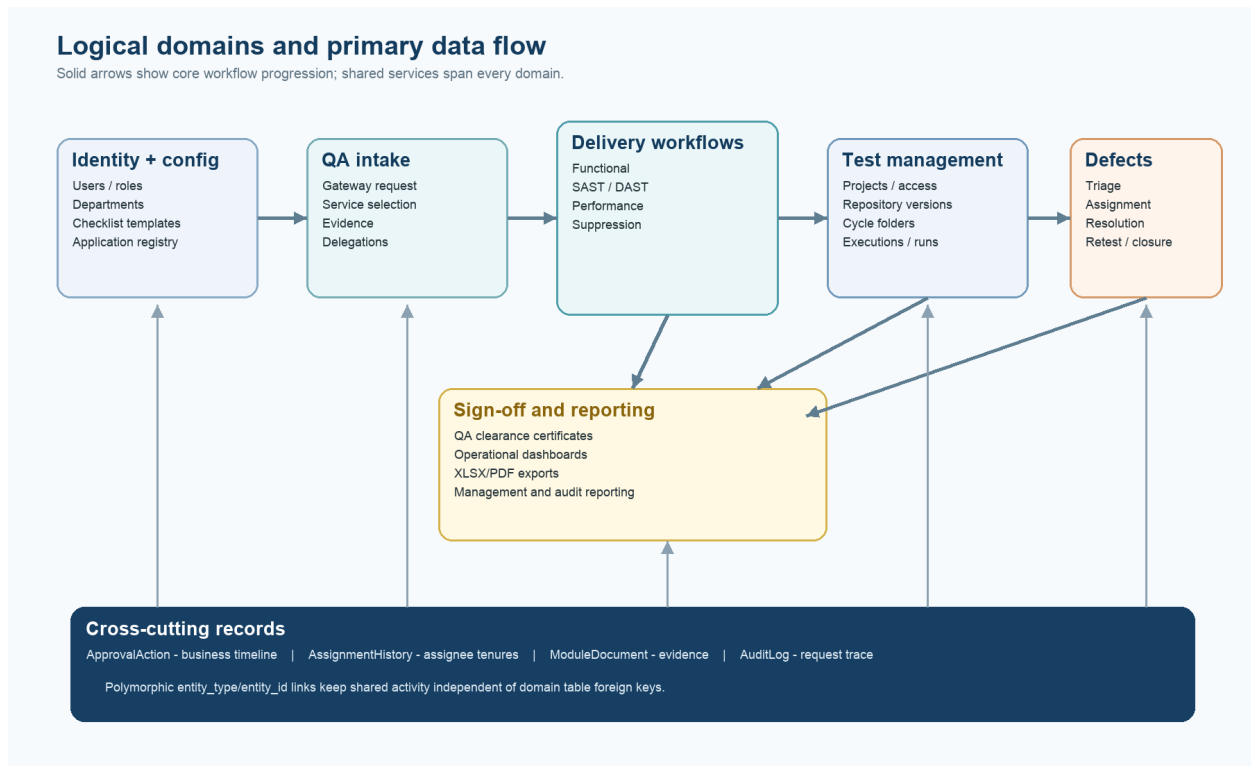


Figure 2. Current logical domains, workflow progression and cross-cutting records.

4. Core business workflows

QA intake gateway

Draft request captures application/change context and requested service types. On raise, linked Functional, SAST, DAST and/or Performance child requests are synchronized. Application-name governance can block or return downstream approval.

Functional delivery

SM and Department Head approval → QA lead readiness → planning/tester assignment → design/execution → defect/retest → QA completion/sign-off → requester verification.

Security testing

SM and Department Head approval → Security Lead readiness → analyst configuration/scanning → finding validation/remediation/rescan → report ready/closed. SAST components and DAST targets/findings remain child records.

Performance testing

Business approvals → QA readiness/feasibility → planning and tester assignment → environment/script/baseline/load execution → analysis → defect-fix-retest → report/sign-off/requester verification.

Test repository

Stable TestCase owns project/folder identity. Each edit/review cycle occurs on TestCaseVersion. Reviewer and QA management decisions are stored on the version and mirrored to TestCase for list compatibility.

Execution and defects

TestCycle contains TestExecution slots. TestExecutionRun preserves attempts. Failed/blocked outcomes link to Defect; governed defect statuses control retest and lifecycle gates.

5. Database design principles

- Oracle identity-safe primary keys are used through a shared pk helper; business IDs are generated separately.
- Foreign keys express core ownership, while selected cross-module records use module/entity identifiers for polymorphic activity and documents.
- Testcase revision data is immutable after governance decisions; a new draft is created from an approved or rejected baseline.
- Current assignment fields are retained on completed requests and support workload/history reporting, but explicit reassignment history is carried by ApprovalAction rather than a dedicated assignment-history table.
- Shared document bytes are not BLOBs. qap_module_documents stores metadata and relative paths into the persistent upload volume.
- Oracle schema evolution is Alembic-managed. Deployments run alembic upgrade head before API startup; autogenerated Oracle DDL, constraints, indexes and data changes require review before release.

6. Complete table catalog

Identity & configuration

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_application_master	ApplicationMaster	Governed application-name registry and Application Owner decision state.	id, name, status, department, requested_by_id, qa_request_id, app_owner_decided_by_id, app_owner_decided_at ...	qap_requests, qap_users
qap_checklist_template_items	ChecklistTemplateItem	Administrator-managed readiness checklist templates by request type.	id, module, item, detail, is_mandatory, sort_order, active, created_at ...	—
qap_departments	Department	Governed department reference data and activation state.	id, name, is_active, created_at	—
qap_id_counters	IdCounter	Concurrency-safe business identifier counters partitioned by prefix and date.	prefix, counter_date, next_value	—
qap_system_settings	SystemSetting	Persisted platform configuration values managed through governance APIs.	id, key, value, updated_at	—
qap_user_departments	UserDepartment	Multi-department membership used for visibility, routing and same-department approval checks.	id, user_id, department, created_at	qap_users

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_user_roles	UserRole	Many-to-one role memberships used by endpoint and workflow authorization.	id, user_id, role	qap_users
qap_users	User	User identity, authentication mode, contact profile, primary department and activation/provisioning state.	id, username, full_name, email, department, login_type, hashed_password, is_active ...	—

QA intake & functional

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_functional_requests	FunctionalRequest	Functional, sanity, regression and UAT workflow state, ownership and linked sign-off.	id, request_id, status, needs_dept_head_reapproval, priority, risk_rating, requester_id, department_head_id ...	qap_requests, qap_signoffs, qap_users
qap_readiness_checklist_items	ReadinessChecklistItem	Functional readiness declarations and QA verification evidence.	id, functional_request_id, item, owner, is_mandatory, requester_checked, is_complete, approved_by_id ...	qap_functional_requests, qap_users
qap_request_delegations	QAResultDelegation	Temporary request-edit delegation, return/recall state and audit timestamps.	id, qa_request_id, target_type, target_id, assigned_by_id, assigned_to_id, assignment_reason, status ...	qap_requests, qap_users
qap_request_documents	QAResultDocument	Dedicated QA intake evidence metadata; bytes live in the upload filesystem.	id, qa_request_id, file_name, stored_path, content_type, file_size, uploaded_by_id, uploaded_at	qap_requests, qap_users
qap_requests	QAResult	QA intake gateway that captures requested services and anchors specialized child requests.	id, request_id, request_date, department, application_name, application_owner, cr_number, epic_number ...	qap_application_master, qap_requests, qap_users

Security & performance

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_dast_checklist_items	DASTChecklistItem	DAST readiness declarations and QA verification evidence.	id, dast_request_id, item, owner, is_mandatory, requester_checked, is_complete, approved_by_id ...	qap_dast_requests, qap_users
qap_dast_findings	DASTFinding	Request-level DAST findings and validation disposition.	id, dast_request_id, issue_id, severity, description, status	qap_dast_requests
qap_dast_requests	DASTRequest	Dynamic application security testing request, assignment and lifecycle state.	id, request_id, risk_category, priority, supporting_docs_path, status, needs_dept_head_reapproval, requester_id ...	qap_requests, qap_users
qap_dast_targets	DASTTarget	URLs/endpoints and environments included in a DAST request.	id, dast_request_id, application_url, environment, authentication_required, test_credentials	qap_dast_requests
qap_performance_checklist_items	PerformanceChecklistItem	Performance readiness declarations and QA verification evidence.	id, performance_request_id, item, data_required, is_mandatory, requester_checked, is_complete, approved_by_id ...	qap_performance_requests, qap_users
qap_performance_requests	PerformanceRequest	Performance-testing workflow, assignment, stages, result analysis and closure state.	id, request_id, application_name, epic_number, cr_number, tool_used, target_load, environment ...	qap_requests, qap_users
qap_sast_checklist_items	SASTChecklistItem	SAST readiness declarations and QA verification evidence.	id, sast_request_id, item, owner, is_mandatory, requester_checked, is_complete, approved_by_id ...	qap_sast_requests, qap_users

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_sast_components	SASTComponent	Repositories/components and source revisions included in a SAST request.	id, sast_request_id, repository_url, git_branch, commit_id, technology_stack, build_number	qap_sast_requests
qap_sast_findings	SASTFinding	Request-level SAST findings and validation disposition.	id, sast_request_id, issue_id, severity, description, status	qap_sast_requests
qap_sast_requests	SASTRequest	Static application security testing request, assignment and lifecycle state.	id, request_id, application_name, epic_number, cr_number, risk_category, priority, source_code_path ...	qap_requests, qap_users
qap_security_scan_results	SecurityScanResult	Imported Fortify scan snapshots, severity totals and source identifiers.	id, request_type, request_id, application_name, application_version, provider, provider_version_id, critical_count ...	qap_users
qap_suppression_items	SuppressionItem	Individual findings covered by a suppression request.	id, suppression_request_id, issue_id, severity, description, justification	qap_suppression_requests
qap_suppression_requests	SuppressionRequest	False-positive/suppression approval workflow linked to SAST or DAST work.	id, suppression_id, application_name, scan_type, department, application_owner, sast_request_id, dast_request_id ...	qap_dast_requests, qap_sast_requests, qap_users

Governance & evidence

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_approval_actions	ApprovalAction	Append-only polymorphic business timeline for decisions, comments and lifecycle events.	id, entity_type, entity_id, step_name, actor_id, actor_role, decision, comments ...	qap_users
qap_assignment_history	AssignmentHistory	Normalized assignee tenures with assignment role, start/end timestamps, actors and reasons.	id, entity_type, entity_id, assignment_role, assignee_id, assigned_by_id, assigned_at, assignment_reason ...	qap_users
qap_audit_logs	AuditLog	Request/security audit trail written through the dedicated audit connection pool.	id, event_type, action, outcome, actor_id, actor_username, actor_name, actor_roles ...	qap_users
qap_module_documents	RequestDocument	Shared evidence metadata keyed by module/entity; file bytes remain on persistent storage.	id, module, request_id, file_name, stored_path, content_type, file_size, uploaded_by_id ...	qap_users
qap_signoffs	QASignOff	QA clearance/sign-off certificate content, electronic approval chain and issuance state.	id, certificate_id, certificate_date, certificate_type, testing_type, testing_request_id, change_request_ids, application_name ...	qap_users

Test management

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_test_case_tags	TestCaseTag	Normalized tags attached to a testcase.	id, test_case_id, tag	qap_test_cases
qap_test_case_version_steps	TestCaseVersionStep	Ordered execution steps belonging to one exact testcase version.	id, version_id, step_no, step_text, expected_result	qap_test_case_versions
qap_test_case_versions	TestCaseVersion	Immutable governed testcase snapshots, review assignments and version decisions.	id, test_case_id, version_major, version_minor, status, epic_id, cr_number, feature_id ...	qap_test_case_versions, qap_test_cases, qap_users
qap_test_cases	TestCase	Stable testcase identity, current pointers and list-compatible mirrors of governed content.	id, test_case_key, project_id, folder_id, epic_id, cr_number, feature_id, user_story_id ...	qap_test_case_versions, qap_test_folders, qap_test_projects, qap_users

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_test_cycle_child_links	TestCycleChildRequestLink	Explicit links between a cycle and specialized QA child requests.	id, cycle_id, child_type, child_id, child_key	qap_test_cycles
qap_test_cycle_folder_access	TestCycleFolderAccess	User/department access grants for a restricted cycle folder.	id, folder_id, department, user_id, granted_by_id, created_at	qap_test_cycle_folders, qap_users
qap_test_cycle_folders	TestCycleFolder	Execution-side cycle folders scoped to a test project.	id, project_id, name, created_by_id, created_at	qap_test_projects, qap_users
qap_test_cycles	TestCycle	Execution set, environment/build context, lifecycle status and QA-request linkage.	id, cycle_key, project_id, name, description, status, start_date, end_date ...	qap_test_cycle_folders, qap_test_projects, qap_users
qap_test_execution_runs	TestExecutionRun	Append-only execution attempts with actual result, evidence context and timestamps.	id, execution_id, attempt_no, status, actual_result, test_run_artifacts, defect_id, executed_by_id ...	qap_test_executions, qap_users
qap_test_executions	TestExecution	One testcase/version slot in a cycle, including assignment and current outcome.	id, cycle_id, test_case_id, pinned_version_id, status, actual_result, test_run_artifacts, defect_id ...	qap_test_case_versions, qap_test_cases, qap_test_cycles, qap_users
qap_test_folders	TestFolder	Hierarchical repository folders within a test project.	id, project_id, parent_id, name, created_by_id, created_at	qap_test_folders, qap_test_projects, qap_users
qap_test_project_members	TestProjectMember	Project membership and project-specific operating role.	id, project_id, user_id, project_role, added_by_id, added_at	qap_test_projects, qap_users
qap_test_project_view_grants	TestProjectViewGrant	Additional user/department read access to otherwise restricted projects.	id, project_id, department, user_id, granted_by_id, created_at	qap_test_projects, qap_users
qap_test_projects	TestProject	Test-management access boundary, defaults, lifecycle and application/department ownership.	id, project_key, name, application_master_id, department, description, is_active, owner_id ...	qap_application_master, qap_users
qap_test_run_defects	TestRunDefect	Lightweight run-level external defect references used by execution gates.	id, run_id, defect_key, defect_url, title, defect_status, notes, linked_by_id ...	qap_test_execution_runs, qap_users
qap_test_steps	TestStep	Legacy/current testcase steps associated with the stable testcase record.	id, test_case_id, step_no, step_text, expected_result	qap_test_cases

Defects

Table	ORM model	How it is used	Significant columns	Foreign-key targets
qap_defect_case_links	DefectTestCaseLink	Many-to-many traceability between governed defects and testcases.	id, defect_id, test_case_id	qap_defects, qap_test_cases
qap_defect_execution_links	DefectExecutionLink	Many-to-many traceability between defects and cycle execution records/runs.	id, defect_id, execution_id, linked_by_id, created_at	qap_defects, qap_test_executions, qap_users
qap_defects	Defect	Governed defect identity, triage, assignment, resolution, retest and closure evidence.	id, defect_key, title, description, status, qa_request_id, cycle_id, primary_test_case_id ...	qap_defects, qap_requests, qap_test_cases, qap_test_cycles, qap_test_executions, qap_users

7. High-significance table relationships

Parent / anchor	Child / dependent	Relationship and usage
qap_requests	functional/sast/dast/performance requests	One gateway may create one or more specialized child workflows; qa_request_id preserves traceability.
qap_test_projects	folders, cases, cycles, members, view grants	Project is the access and organizational boundary for test management.
qap_test_cases	test_case_versions	Stable logical identity to many version snapshots; current draft/approved pointers select active content.
qap_test_case_versions	test_case_version_steps	Each snapshot owns its own ordered steps; comparison is version-to-version.
qap_test_cycles	qap_test_executions	A cycle contains testcase execution slots and assignment/result state.
qap_test_executions	qap_test_execution_runs	Current slot to append-only attempts; supports evidence and historical outcomes.
qap_defects	qap_defect_case_links / qap_test_run_defects	Defect traces to cases and execution context; snapshot table supports execution gates/reporting.
Any governed entity	qap_approval_actions	Polymorphic entity_type/entity_id activity timeline; no database FK by design.
Any module/item	qap_module_documents	Polymorphic module/request_id metadata; physical file stored outside Oracle.
qap_users	roles, departments, assignments, actions	User identity is referenced by governance, ownership, assignments, and audit.

8. Testcase versioning in detail

TestCase is the durable record and human-readable key. TestCaseVersion stores the governed content snapshot, version numbers, review decisions and author/approver assignments. TestCaseVersionStep stores the steps belonging to that exact snapshot.

- Minor approval increments the minor component: 1.9 → 1.10.
- Major approval is explicitly selected at final QA approval and requires justification: 1.9 → 2.0.
- Only the newest version is current in the UI; older history is displayed as Superseded without destroying its original decision metadata.
- Version comparison reads two snapshots and returns changed content fields plus added, removed or modified steps.
- Approved/rejected history is governed and cannot be hard-deleted through normal repository operations.

9. Security and authorization model

Control	Implementation	Important consequence
Authentication	JWT bearer token; local password or LDAP verification	API reloads the active user, roles and departments; disabling a user takes effect independently of token claims.
Authorization	Role constants plus endpoint dependencies and inline business checks	UI gating is advisory; backend remains authoritative.
Department scope	Multi-department membership and dashboard_department_scope	Business users see their departments; QA/executive roles have deliberate organization-wide review visibility.
Self-approval prevention	Requester/author identity checked at decision points	Prevents a person approving their own governed record unless explicit admin bypass applies.
Assignment control	Current assignee/lead/tester plus lifecycle status	Who can act changes with workflow stage and assignment.
Confidential roles	Redacted from non-admin user-list serialization	Sensitive access role is not exposed in ordinary pickers/API responses.
Request audit	Correlation ID, status, duration, actor snapshot	Supports operational investigation without mixing with business workflow activity.

10. Documents, evidence and rich content

Gateway evidence uses `qap_request_documents`. Most other modules use `qap_module_documents` through shared helpers. The module value distinguishes request-level, checklist-item, defect and comment-image records. Files are written below the configured upload root using request-first folders and collision-safe names.

- Database backup alone is insufficient; restore requires a consistent upload-volume backup.
- Deleting metadata without deleting bytes creates orphans; deleting bytes without metadata produces broken downloads.
- Document authorization is workflow-stage-specific and enforced by each router before shared storage helpers are invoked.
- Uploads/removals may append `ApprovalAction` entries so evidence changes are visible in the business timeline.

11. Audit, cache and observability

Facility	Data/store	Behavior
Business activity	<code>qap_approval_actions</code>	Human-readable workflow decisions, state changes, comments and document events.
Platform audit	<code>qap_audit_logs</code> via <code>AuditSessionLocal</code>	Background request/security audit using a separate Oracle pool.

Facility	Data/store	Behavior
Cache	Redis qa_portal:* namespace	JSON caches and prefix invalidation; cache failure becomes a miss, not a request failure.
Health	/api/health	Checks Oracle live and reports last-known Redis availability.
Correlation	X-Request-ID / X-Audit-Request-ID	Connects client-visible failures to persisted request audit and logs.
Timing	Server-Timing header	Returns application duration for browser/edge diagnosis.

12. Deployment and capacity considerations

- Docker Compose runs Nginx frontend, multi-worker FastAPI backend and Redis. Oracle is external/reachable through DATABASE_URL.
- Connection pools are per worker. Maximum potential sessions are approximately workers × (pool_size + max_overflow), plus the separate audit pools. Tune against Oracle PROCESSES/SESSIONS limits.
- The upload volume must be shared by every API replica. A node-local volume is unsafe for horizontal scaling unless requests remain sticky and failover limitations are accepted.
- Redis is optional for correctness but required for cross-worker cache coherence and reliable distributed startup locks.
- DB_QUERY_TIMEOUT_MS sets python-oracledb call_timeout. Validate reporting workloads before reducing it.
- Trusted proxy CIDRs must match the actual reverse-proxy network so client IP audit data cannot be spoofed.

13. Significant risks and recommended controls

Priority	Risk / limitation	Why significant	Recommended control
High	Unreviewed Alembic revisions	Generated Oracle DDL can still be destructive or require an explicit data backfill.	Review upgrade/downgrade logic, test on a disposable Oracle schema, then run alembic upgrade head.
High	DB and file backup consistency	Evidence bytes and metadata are separate stores.	Snapshot Oracle and upload volume under one recoverable backup procedure; test restores.
High	Pool multiplication by workers	Default pool settings can exceed Oracle session limits.	Capacity-test and calculate aggregate main + audit sessions before scaling workers.

Priority	Risk / limitation	Why significant	Recommended control
Resolved	Normalized assignment tenure history	qap_assignment_history records each assignee role, start/end time, assigning and unassigning actor, and reasons independently of the current-assignee fields.	Maintain the Alembic-managed table and use GET /api/approvals/assignment-history for scoped management and audit reporting.
Medium	Polymorphic entity references	ApprovalAction and RequestDocument cannot use conventional FKs across many entity types.	Centralize module/entity constants; add integrity audit jobs for orphan detection.
Medium	Status strings across modules	Many workflows use string states and mirrored frontend constants.	Add automated parity/state-machine tests and consider database check constraints where safe.
Medium	Redis degradation is silent by design	Correctness remains, but latency and duplicate startup work may rise.	Alert on cache availability and cache miss rates; document degraded mode.
Low	TestCase mirror duplication	TestCase mirrors current version content for compatibility.	Keep all mutations through repository helpers and add consistency checks.

14. Change-impact guide

Change type	Inspect first	Also verify
Add workflow status	backend constants/router transitions	Frontend constants, badges, filters, dashboards, terminal-state logic and activity views.
Add database column	ORM model + Oracle migration script	Pydantic schema, TypeScript type, create/edit/detail UI, exports, audit snapshots.
Add attachment category	documents.py module key and authorization	Upload path, qap_module_documents, activity logging, backup/retention.
Change assignment	Domain assignment endpoint	Reassignment authorization, change detection, ApprovalAction, workload dashboard.
Change testcase versioning	TestCaseVersion creation/approval helpers	Mirror synchronization, compare API, cycle pinning, deletion governance.
Add dashboard metric	Scoped query and cache key	Department scope, invalidation module list, date filter semantics, index needs.
Scale API workers	WEB_CONCURRENCY	Aggregate Oracle sessions, Redis locks, shared uploads, background audit pool.

15. Operational verification checklist

- Health endpoint reports Oracle reachable and expected Redis state.
- Schema migration scripts have been applied and verified against the target Oracle schema.
- Aggregate main/audit connection ceiling is within Oracle capacity.
- Upload volume is mounted, writable, backed up and shared across replicas.
- LDAP, local login, disabled-user and JIT department-selection paths are tested.
- Role/department/self-approval scenarios are validated with representative users.
- Core state transitions are tested for Functional, SAST, DAST, Performance, Test Repository, Execution and Defects.
- Audit correlation ID can be traced from response header to application log and qap_audit_logs.
- Dashboard cache invalidates after successful domain writes.
- Testcase version history, Superseded display and two-version comparison are verified.

Appendix A. Source map

Area	Primary source files
Application composition	frontend/src/App.tsx, frontend/src/components/Layout.tsx
Shared UI/workflow controls	frontend/src/components/Common.tsx
API composition and middleware	backend/app/main.py
Database engines and pools	backend/app/database.py
ORM/data model	backend/app/models.py
Schemas/contracts	backend/app/schemas.py, frontend/src/types.ts
Authentication and scope	backend/app/auth.py, backend/app/deps.py, backend/app/routers/auth.py
Caching	backend/app/cache.py
Evidence storage	backend/app/documents.py, backend/app/storage_config.py
Domain routes	backend/app/routers/*.py
Deployment	docker-compose.yml, frontend/nginx.conf, backend/Dockerfile, frontend/Dockerfile
Schema change history	backend/alembic/versions/*.py, backend/MIGRATIONS.md

Appendix B. Interpretation notes

“Uses” in the table catalog means application ownership inferred from ORM definitions and route behavior, not observed production query frequency. The repository contains extensive migration commentary; this document summarizes the current implementation rather than reproducing historical change notes.

Generated on 23 August 2026.

Current implementation update - 23 August 2026

This section records the application behavior delivered since the prior architecture baseline. It describes implemented behavior only; future deployment patterns and unimplemented services are excluded.

User experience and workflow controls

- A global success-toast center confirms authenticated create, update, assignment, upload, decision, removal, restore and submission mutations without replacing validation/error dialogs.
- Shared confirmation and decision dialogs are used for consequential workflow and destructive actions; Return and Reject require non-blank remarks at both API-schema and user-interface boundaries.
- Functional, SAST, DAST and Performance use the same readiness-pass error dialog and the same supporting-evidence control, including upload counts, delete confirmation, lock messaging and post-save refresh.
- Request delegation uses a common active-user picker and retains requester ownership; assignment, return and recall actions remain auditable.
- Pending Approvals is server-paginated, category-filtered and grouped beneath its parent QA Request or Test Project. Terminal records do not display a meaningless Pending With placeholder.

Data presentation, collaboration and reporting

- Dashboard attention cards provide complete metric definitions and open consolidated, server-paginated drill-down records for Active Projects, Open Security Findings, Waiting for a Decision and Active Child Requests.
- Jira-style rich text preserves pasted table row/column merges through the encoded content model, browser rendering and PDF export.
- Repository, request and supporting-evidence layouts reuse shared presentation controls; newly uploaded evidence is reloaded after Save Changes.
- Assignment tenure is normalized in `qap_assignment_history`, while current-assignee fields continue to provide fast operational lookup.
- The browser manifest/favicon set and the application footer are included in the current frontend shell.

Current Docker Compose deployment

Service	Current responsibility	Runtime dependency
migrate	One-shot alembic upgrade head before API startup.	Oracle and reviewed Alembic revisions.
backend	FastAPI/Uvicorn API with configurable worker and Oracle pool sizing.	Oracle, Redis health, upload volume and log volume.
redis	Shared response cache and startup/distributed lock coordinator with append-only persistence.	Docker-managed Redis volume.
frontend	Nginx HTTPS termination, static React SPA delivery and /api reverse proxy.	Backend health and mounted TLS certificates.

Deployment boundary Oracle remains external to docker-compose.yml. Retained evidence uses the deployment-controlled UPLOAD_STORAGE_ROOT backed by qa_portal_uploads or UPLOAD_STORAGE_HOST_PATH. Schema changes are Alembic-managed and the migrate service must complete successfully before the backend starts.

Appendix C. Detailed database dictionary

This appendix is generated from SQLAlchemy metadata in backend/app/models.py. It records the 50 mapped tables and all 599 mapped columns, including logical type, nullability, keys, defaults and implementation meaning. Alembic revisions are the versioned schema-change authority; the deployed Oracle schema remains the runtime authority.

Domain	Tables	Columns	Purpose
Identity, configuration and governance	12	102	See per-table entries below.
QA intake and functional workflow	6	101	See per-table entries below.
Security and performance workflows	13	158	See per-table entries below.
Test repository and execution	16	172	See per-table entries below.
Defect management	3	66	See per-table entries below.

Identity, configuration and governance

gap_application_master

ApplicationMaster. Governed application-name registry and Application Owner decision state.

13 columns; primary key: id; 4 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
name	VARCHAR(150)	NOT NULL; UNIQUE	Name.
status	VARCHAR(20)	NULL; INDEX; default=PENDING_APP_OWNER	Current governed lifecycle state.
department	VARCHAR(150)	NULL	Department.
requested_by_id	INTEGER	FK→gap_users.id; NULL	Reference to the associated requested by record.
qa_request_id	INTEGER	FK→gap_requests.id; NULL	Reference to the associated qa request record.
app_owner_decided_by_id	INTEGER	FK→gap_users.id; NULL	Reference to the associated app owner decided by record.
app_owner_decided_at	DATETIME	NULL	Timestamp for app owner decided.
app_owner_comments	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
decided_by_id	INTEGER	FK→gap_users.id; NULL	Reference to the associated decided by record.
decided_at	DATETIME	NULL	Timestamp for decided.
comments	TEXT	NULL	Decision remarks or audit context.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: name | Index ix_qap_application_master_status: status

qap_approval_actions

ApprovalAction. Append-only polymorphic business timeline for decisions, comments and lifecycle events.

11 columns; primary key: id; 1 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
entity_type	VARCHAR(32)	NULL	Entity type.
entity_id	INTEGER	NULL; INDEX	Reference to the associated entity record.
step_name	VARCHAR(64)	NULL	Step name.
actor_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated actor record.
actor_role	VARCHAR(150)	NULL	Actor role.
decision	VARCHAR(64)	NULL	Decision.
comments	TEXT	NULL	Decision remarks or audit context.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
previous_state	VARCHAR(30)	NULL	Previous state.
new_state	VARCHAR(30)	NULL	New state.

Constraints and indexes: Index ix_qap_appact_entity_created: entity_type, entity_id, created_at | Index ix_qap_approval_actions_entity_id: entity_id

qap_assignment_history

AssignmentHistory. Normalized assignee tenures with assignment role, start/end timestamps, actors and reasons.

11 columns; primary key: id; 3 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
entity_type	VARCHAR(32)	NOT NULL	Entity type.
entity_id	INTEGER	NOT NULL	Reference to the associated entity record.
assignment_role	VARCHAR(40)	NOT NULL	Assignment role.
assignee_id	INTEGER	FK→qap_users.id; NOT NULL	Reference to the associated assignee record.
assigned_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assigned by record.
assigned_at	DATETIME	NOT NULL; default=now	Timestamp for assigned.
assignment_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
unassigned_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated unassigned by record.
unassigned_at	DATETIME	NULL	Timestamp for unassigned.
unassignment_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.

Constraints and indexes: Check: unassigned_at IS NULL OR unassigned_at >= assigned_at | Index ix_qap_asgh_entity_active: entity_type, entity_id, assignment_role, unassigned_at | Index ix_qap_asgh_user_time: assignee_id, assigned_at

qap_audit_logs

AuditLog. Request/security audit trail written through the dedicated audit connection pool.

19 columns; primary key: id; 1 foreign keys; 9 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
event_type	VARCHAR(40)	NOT NULL; INDEX	Event type.
action	VARCHAR(80)	NOT NULL; INDEX	Action.
outcome	VARCHAR(20)	NOT NULL; INDEX	Outcome.
actor_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated actor record.
actor_username	VARCHAR(150)	NULL; INDEX	Actor username.
actor_name	VARCHAR(150)	NULL	Actor name.
actor_roles	VARCHAR(500)	NULL	Actor roles.
method	VARCHAR(10)	NULL	Method.
path	VARCHAR(500)	NULL; INDEX	Path.
status_code	INTEGER	NULL	State/disposition value for status code.
target_type	VARCHAR(64)	NULL	Target type.
target_id	VARCHAR(100)	NULL	Reference to the associated target record.
target_name	VARCHAR(255)	NULL	Target name.
details	TEXT	NULL	Details.
ip_address	VARCHAR(64)	NULL	Ip address.
user_agent	VARCHAR(500)	NULL	User agent.
request_id	VARCHAR(64)	NULL; INDEX	Human-readable business request identifier.
created_at	DATETIME	NOT NULL; INDEX; default=now	Server-side creation timestamp.

Constraints and indexes: Index ix_qap_audit_actor_created: actor_id, created_at | Index ix_qap_audit_logs_action: action | Index ix_qap_audit_logs_actor_username: actor_username | Index ix_qap_audit_logs_created_at: created_at | Index ix_qap_audit_logs_event_type: event_type | Index ix_qap_audit_logs_outcome: outcome | Index ix_qap_audit_logs_path: path | Index ix_qap_audit_logs_request_id: request_id | Index ix_qap_audit_target: target_type, target_id

qap_checklist_template_items

ChecklistTemplateItem. Administrator-managed readiness checklist templates by request type.

9 columns; primary key: id.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
module	VARCHAR(20)	NOT NULL	Module.
item	VARCHAR(255)	NOT NULL	Item.
detail	VARCHAR(255)	NULL	Detail.
is_mandatory	BOOLEAN	NULL; default=False	Boolean flag indicating mandatory.
sort_order	INTEGER	NULL; default=0	Sort order.
active	BOOLEAN	NULL; default=True	Active.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: module, item

qap_departments

Department. Governed department reference data and activation state.

4 columns; primary key: id.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
name	VARCHAR(150)	NOT NULL; UNIQUE	Name.
is_active	BOOLEAN	NULL; default=True	Whether the record is active/available for normal use.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: name

qap_id_counters

IdCounter. Concurrency-safe business identifier counters partitioned by prefix and date.

3 columns; primary key: prefix, counter_date.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
prefix	VARCHAR(20)	PK; NOT NULL	Prefix.
counter_date	DATE	PK; NOT NULL	Counter date.
next_value	INTEGER	NOT NULL	Next value.

qap_module_documents

RequestDocument. Shared evidence metadata keyed by module/entity; file bytes remain on persistent storage.

9 columns; primary key: id; 1 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
module	VARCHAR(20)	NOT NULL	Module.
request_id	INTEGER	NOT NULL; INDEX	Human-readable business request identifier.
file_name	VARCHAR(255)	NOT NULL	File name.
stored_path	VARCHAR(500)	NOT NULL	Stored path.
content_type	VARCHAR(150)	NULL	Content type.
file_size	INTEGER	NULL	File size.
uploaded_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated uploaded by record.
uploaded_at	DATETIME	NULL; default=now	Timestamp for uploaded.

Constraints and indexes: Index ix_qap_moddocs_req_uploaded: module, request_id, uploaded_at | Index ix_qap_module_documents_request_id: request_id

qap_system_settings

SystemSetting. Persisted platform configuration values managed through governance APIs.

4 columns; primary key: id.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
key	VARCHAR(100)	NOT NULL; UNIQUE	Key.
value	TEXT	NOT NULL	Value.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: key

qap_user_departments

UserDepartment. Multi-department membership used for visibility, routing and same-department approval checks.

4 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
user_id	INTEGER	FK→qap_users.id; NOT NULL	Reference to the associated user record.
department	VARCHAR(150)	NOT NULL	Department.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: user_id, department

qap_user_roles

UserRole. Many-to-one role memberships used by endpoint and workflow authorization.

3 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
user_id	INTEGER	FK→qap_users.id; NOT NULL	Reference to the associated user record.
role	VARCHAR(32)	NOT NULL	Role.

Constraints and indexes: Unique: user_id, role

qap_users

User. User identity, authentication mode, contact profile, primary department and activation/provisioning state.

12 columns; primary key: id.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
username	VARCHAR(64)	NOT NULL; UNIQUE	Username.
full_name	VARCHAR(150)	NOT NULL	Full name.
email	VARCHAR(150)	NULL	Email.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
department	VARCHAR(150)	NULL	Department.
login_type	VARCHAR(16)	NOT NULL; default=STANDARD	Login type.
hashed_password	VARCHAR(255)	NULL	Hashed password.
is_active	BOOLEAN	NULL; default=True	Whether the record is active/available for normal use.
needs_role_review	BOOLEAN	NULL; default=False	Needs role review.
needs_department_selection	BOOLEAN	NULL; default=False	Needs department selection.
admin_managed_only	BOOLEAN	NULL; default=False	Admin managed only.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: username

QA intake and functional workflow

gap_functional_requests

FunctionalRequest. Functional, sanity, regression and UAT workflow state, ownership and linked sign-off.

14 columns; primary key: id; 5 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
request_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Human-readable business request identifier.
status	VARCHAR(32)	NULL; default=DRAFT	Current governed lifecycle state.
needs_dept_head_reapproval	BOOLEAN	NULL; default=False	Needs dept head reapproval.
priority	VARCHAR(16)	NULL	Priority.
risk_rating	VARCHAR(16)	NULL	Risk rating.
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
department_head_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated department head record.
qa_lead_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated qa lead record.
assigned_tester_ids	VARCHAR(255)	NULL	Assigned tester ids.
signoff_id	INTEGER	FK→qap_signoffs.id; NULL	Reference to the associated signoff record.
qa_request_id	INTEGER	FK→qap_requests.id; NULL	Reference to the associated qa request record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: request_id | Index ix_qap_func_qa_req: qa_request_id | Index ix_qap_func_status_created: status, created_at

gap_readiness_checklist_items

ReadinessChecklistItem. Functional readiness declarations and QA verification evidence.

9 columns; primary key: id; 2 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
functional_request_id	INTEGER	FK→qap_functional_requests.id; NULL	Reference to the associated functional request record.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
item	VARCHAR(255)	NOT NULL	Item.
owner	VARCHAR(150)	NULL	Owner.
is_mandatory	BOOLEAN	NULL; default=True	Boolean flag indicating mandatory.
requester_checked	BOOLEAN	NULL; default=False	Requester checked.
is_complete	BOOLEAN	NULL; default=False	Boolean flag indicating complete.
approved_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated approved by record.
approved_at	DATETIME	NULL	Timestamp for approved.

Constraints and indexes: Index ix_qap_readiness_func_req: functional_request_id

qap_request_delegations

QARequestDelegation. Temporary request-edit delegation, return/recall state and audit timestamps.

12 columns; primary key: id; 4 foreign keys; 3 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
qa_request_id	INTEGER	FK→qap_requests.id; NOT NULL	Reference to the associated qa request record.
target_type	VARCHAR(24)	NOT NULL; default=QA_REQUEST	Target type.
target_id	INTEGER	NOT NULL	Reference to the associated target record.
assigned_by_id	INTEGER	FK→qap_users.id; NOT NULL	Reference to the associated assigned by record.
assigned_to_id	INTEGER	FK→qap_users.id; NOT NULL	Reference to the associated assigned to record.
assignment_reason	TEXT	NOT NULL	Human-entered rationale retained for workflow/audit traceability.
status	VARCHAR(16)	NOT NULL; default=ACTIVE	Current governed lifecycle state.
assigned_at	DATETIME	NOT NULL; default=now	Timestamp for assigned.
closed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated closed by record.
returned_at	DATETIME	NULL	Timestamp for returned.
return_comments	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.

Constraints and indexes: Check: target_type IN ('QA_REQUEST','FUNCTIONAL','SAST','DAST','PERFORMANCE') | Check: status IN ('ACTIVE','RETURNED','RECALLED') | Index ix_qap_del_req_status: qa_request_id, status | Index ix_qap_del_target_status: target_type, target_id, status | Index ix_qap_del_user_status: assigned_to_id, status

qap_request_documents

QARequestDocument. Dedicated QA intake evidence metadata; bytes live in the upload filesystem.

8 columns; primary key: id; 2 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
qa_request_id	INTEGER	FK→qap_requests.id; NULL	Reference to the associated qa request record.
file_name	VARCHAR(255)	NOT NULL	File name.
stored_path	VARCHAR(500)	NOT NULL	Stored path.
content_type	VARCHAR(150)	NULL	Content type.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
file_size	INTEGER	NULL	File size.
uploaded_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated uploaded by record.
uploaded_at	DATETIME	NULL; default=now	Timestamp for uploaded.

qap_requests

QAResult. QA intake gateway that captures requested services and anchors specialized child requests.

28 columns; primary key: id; 3 foreign keys; 3 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
request_id	VARCHAR(40)	NULL; UNIQUE	Human-readable business request identifier.
request_date	DATE	NULL; default=<lambda>	Request date.
department	VARCHAR(150)	NULL	Department.
application_name	VARCHAR(150)	NOT NULL	Application name.
application_owner	VARCHAR(150)	NULL	Application owner.
cr_number	VARCHAR(64)	NULL	Cr number.
epic_number	VARCHAR(150)	NULL	Epic number.
change_type	VARCHAR(32)	NULL	Change type.
bug_fix_source_request_id	VARCHAR(40)	FK→qap_requests.request_id; NULL	Reference to the associated bug fix source request record.
vendor_si_partner	VARCHAR(150)	NULL	Vendor si partner.
technology_stack	VARCHAR(150)	NULL	Technology stack.
release_version	VARCHAR(64)	NULL	Release version.
build_number	VARCHAR(64)	NULL	Build number.
environment	VARCHAR(32)	NULL	Environment.
target_promotion_environment	VARCHAR(32)	NULL	Target promotion environment.
request_types	VARCHAR(255)	NULL	Request types.
request_type_other	VARCHAR(150)	NULL	Request type other.
target_release_date	DATE	NULL	Target release date.
supporting_doc_path	VARCHAR(255)	NULL	Supporting doc path.
remarks	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
change_description	TEXT	NULL	Change description.
draft_child_details	TEXT	NULL	Draft child details.
status	VARCHAR(32)	NULL; INDEX; default=DRAFT	Current governed lifecycle state.
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
application_master_id	INTEGER	FK→qap_application_master.id; NULL	Reference to the associated application master record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: request_id | Index ix_qap_req_bugfix_source: bug_fix_source_request_id | Index ix_qap_req_dept_status_created: department, status, created_at | Index ix_qap_requests_status: status

qap_signoffs

QASignOff. QA clearance/sign-off certificate content, electronic approval chain and issuance state.

30 columns; primary key: id; 5 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
certificate_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Reference to the associated certificate record.
certificate_date	DATE	NULL; default=<lambda>	Certificate date.
certificate_type	VARCHAR(32)	NULL	Certificate type.
testing_type	VARCHAR(16)	NULL	Testing type.
testing_request_id	VARCHAR(40)	NULL	Reference to the associated testing request record.
change_request_ids	VARCHAR(255)	NULL	Change request ids.
application_name	VARCHAR(150)	NOT NULL	Application name.
application_owner	VARCHAR(150)	NULL	Application owner.
department	VARCHAR(150)	NULL	Department.
vendor_si_partner	VARCHAR(150)	NULL	Vendor si partner.
technology_stack	VARCHAR(150)	NULL	Technology stack.
risk_tier	VARCHAR(32)	NULL	Risk tier.
release_version	VARCHAR(64)	NULL	Release version.
build_number	VARCHAR(64)	NULL	Build number.
environment_tested	VARCHAR(32)	NULL	Environment tested.
target_promotion_environment	VARCHAR(32)	NULL	Target promotion environment.
validity_from	DATE	NULL	Validity from.
validity_to	DATE	NULL	Validity to.
exit_criteria_notes	TEXT	NULL	Exit criteria notes.
open_defect_summary	TEXT	NULL	Open defect summary.
residual_risk_notes	TEXT	NULL	Residual risk notes.
status	VARCHAR(32)	NULL; default=DRAFT	Current governed lifecycle state.
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
reviewed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated reviewed by record.
approved_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated approved by record.
issued_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated issued by record.
signed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated signed by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: certificate_id | Index ix_qap_signoff_testreq: testing_request_id

Security and performance workflows

qap_dast_checklist_items

DASTChecklistItem. DAST readiness declarations and QA verification evidence.

9 columns; primary key: id; 2 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
dast_request_id	INTEGER	FK→qap_dast_requests.id; NULL	Reference to the associated dast request record.
item	VARCHAR(255)	NOT NULL	Item.
owner	VARCHAR(150)	NULL	Owner.
is_mandatory	BOOLEAN	NULL; default=False	Boolean flag indicating mandatory.
requester_checked	BOOLEAN	NULL; default=False	Requester checked.
is_complete	BOOLEAN	NULL; default=False	Boolean flag indicating complete.
approved_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated approved by record.
approved_at	DATETIME	NULL	Timestamp for approved.

qap_dast_findings

DASTFinding. Request-level DAST findings and validation disposition.

6 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
dast_request_id	INTEGER	FK→qap_dast_requests.id; NULL	Reference to the associated dast request record.
issue_id	VARCHAR(64)	NULL	Reference to the associated issue record.
severity	VARCHAR(16)	NULL	Severity.
description	TEXT	NULL	Description.
status	VARCHAR(32)	NULL; default=Open	Current governed lifecycle state.

qap_dast_requests

DASTRequest. Dynamic application security testing request, assignment and lifecycle state.

14 columns; primary key: id; 4 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
request_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Human-readable business request identifier.
risk_category	VARCHAR(16)	NULL	Risk category.
priority	VARCHAR(16)	NULL	Priority.
supporting_docs_path	VARCHAR(255)	NULL	Supporting docs path.
status	VARCHAR(32)	NULL; default=DRAFT	Current governed lifecycle state.
needs_dept_head_reapproval	BOOLEAN	NULL; default=False	Needs dept head reapproval.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
security_lead_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated security lead record.
security_analyst_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated security analyst record.
report_path	VARCHAR(255)	NULL	Report path.
qa_request_id	INTEGER	FK→qap_requests.id; NULL	Reference to the associated qa request record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: request_id | Index ix_qap_dast_qa_req: qa_request_id | Index ix_qap_dast_status_created: status, created_at

qap_dast_targets

DASTTarget. URLs/endpoints and environments included in a DAST request.

6 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
dast_request_id	INTEGER	FK→qap_dast_requests.id; NOT NULL	Reference to the associated dast request record.
application_url	VARCHAR(2000)	NOT NULL	Application url.
environment	VARCHAR(500)	NULL	Environment.
authentication_required	VARCHAR(16)	NULL; default=No	Authentication required.
test_credentials	VARCHAR(2000)	NULL	Test credentials.

qap_performance_checklist_items

PerformanceChecklistItem. Performance readiness declarations and QA verification evidence.

9 columns; primary key: id; 2 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
performance_request_id	INTEGER	FK→qap_performance_requests.id; NULL	Reference to the associated performance request record.
item	VARCHAR(255)	NOT NULL	Item.
data_required	VARCHAR(255)	NULL	Data required.
is_mandatory	BOOLEAN	NULL; default=True	Boolean flag indicating mandatory.
requester_checked	BOOLEAN	NULL; default=False	Requester checked.
is_complete	BOOLEAN	NULL; default=False	Boolean flag indicating complete.
approved_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated approved by record.
approved_at	DATETIME	NULL	Timestamp for approved.

gap_performance_requests

PerformanceRequest. Performance-testing workflow, assignment, stages, result analysis and closure state.

27 columns; primary key: id; 3 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
request_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Human-readable business request identifier.
application_name	VARCHAR(150)	NOT NULL	Application name.
epic_number	VARCHAR(150)	NULL	Epic number.
cr_number	VARCHAR(64)	NULL	Cr number.
tool_used	VARCHAR(150)	NULL	Tool used.
target_load	VARCHAR(150)	NULL	Target load.
environment	VARCHAR(32)	NULL	Environment.
risk_category	VARCHAR(16)	NULL	Risk category.
priority	VARCHAR(16)	NULL	Priority.
request_type	VARCHAR(120)	NULL	Request type.
change_type	VARCHAR(32)	NULL	Change type.
vendor_si_partner	VARCHAR(150)	NULL	Vendor si partner.
technology_stack	VARCHAR(150)	NULL	Technology stack.
release_version	VARCHAR(64)	NULL	Release version.
build_number	VARCHAR(64)	NULL	Build number.
hash_value	VARCHAR(255)	NULL	Hash value.
target_promotion_environment	VARCHAR(32)	NULL	Target promotion environment.
status	VARCHAR(32)	NULL; default=DRAFT	Current governed lifecycle state.
needs_dept_head_reapproval	BOOLEAN	NULL; default=False	Needs dept head reapproval.
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
engineer_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated engineer record.
assigned_tester_ids	VARCHAR(255)	NULL	Assigned tester ids.
report_path	VARCHAR(255)	NULL	Report path.
qa_request_id	INTEGER	FK→qap_requests.id; NULL	Reference to the associated qa request record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: request_id | Index ix_qap_perf_qa_req: qa_request_id | Index ix_qap_perf_status_created: status, created_at

gap_sast_checklist_items

SASTChecklistItem. SAST readiness declarations and QA verification evidence.

9 columns; primary key: id; 2 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
sast_request_id	INTEGER	FK→qap_sast_requests.id; NULL	Reference to the associated sast request record.
item	VARCHAR(255)	NOT NULL	Item.
owner	VARCHAR(150)	NULL	Owner.
is_mandatory	BOOLEAN	NULL; default=False	Boolean flag indicating mandatory.
requester_checked	BOOLEAN	NULL; default=False	Requester checked.
is_complete	BOOLEAN	NULL; default=False	Boolean flag indicating complete.
approved_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated approved by record.
approved_at	DATETIME	NULL	Timestamp for approved.

qap_sast_components

SASTComponent. Repositories/components and source revisions included in a SAST request.

7 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
sast_request_id	INTEGER	FK→qap_sast_requests.id; NOT NULL	Reference to the associated sast request record.
repository_url	VARCHAR(1000)	NULL	Repository url.
git_branch	VARCHAR(500)	NULL	Git branch.
commit_id	VARCHAR(500)	NULL	Reference to the associated commit record.
technology_stack	VARCHAR(500)	NULL	Technology stack.
build_number	VARCHAR(300)	NULL	Build number.

qap_sast_findings

SASTFinding. Request-level SAST findings and validation disposition.

6 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
sast_request_id	INTEGER	FK→qap_sast_requests.id; NULL	Reference to the associated sast request record.
issue_id	VARCHAR(64)	NULL	Reference to the associated issue record.
severity	VARCHAR(16)	NULL	Severity.
description	TEXT	NULL	Description.
status	VARCHAR(32)	NULL; default=Open	Current governed lifecycle state.

qap_sast_requests

SASTRequest. Static application security testing request, assignment and lifecycle state.

18 columns; primary key: id; 4 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
request_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Human-readable business request identifier.
application_name	VARCHAR(150)	NOT NULL	Application name.
epic_number	VARCHAR(150)	NULL	Epic number.
cr_number	VARCHAR(64)	NULL	Cr number.
risk_category	VARCHAR(16)	NULL	Risk category.
priority	VARCHAR(16)	NULL	Priority.
source_code_path	VARCHAR(255)	NULL	Source code path.
hash_value	VARCHAR(255)	NULL	Hash value.
status	VARCHAR(32)	NULL; default=DRAFT	Current governed lifecycle state.
needs_dept_head_reapproval	BOOLEAN	NULL; default=False	Needs dept head reapproval.
requester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated requester record.
security_lead_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated security lead record.
security_analyst_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated security analyst record.
report_path	VARCHAR(255)	NULL	Report path.
qa_request_id	INTEGER	FK→qap_requests.id; NULL	Reference to the associated qa request record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: request_id | Index ix_qap_sast_qa_req: qa_request_id | Index ix_qap_sast_status_created: status, created_at

qap_security_scan_results

SecurityScanResult. Imported Fortify scan snapshots, severity totals and source identifiers.

16 columns; primary key: id; 1 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
request_type	VARCHAR(8)	NOT NULL; INDEX	Request type.
request_id	INTEGER	NOT NULL; INDEX	Human-readable business request identifier.
application_name	VARCHAR(150)	NOT NULL	Application name.
application_version	VARCHAR(100)	NOT NULL	Application version.
provider	VARCHAR(40)	NOT NULL; default=Fortify SSC	Provider.
provider_version_id	VARCHAR(100)	NOT NULL	Reference to the associated provider version record.
critical_count	INTEGER	NOT NULL; default=0	Stored aggregate count for critical.
high_count	INTEGER	NOT NULL; default=0	Stored aggregate count for high.
medium_count	INTEGER	NOT NULL; default=0	Stored aggregate count for medium.
low_count	INTEGER	NOT NULL; default=0	Stored aggregate count for low.
total_count	INTEGER	NOT NULL; default=0	Stored aggregate count for total.
audit_url	VARCHAR(1000)	NULL	Audit url.
filters_json	TEXT	NULL	Filters json.
imported_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated imported by record.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
imported_at	DATETIME	NOT NULL; default=now	Timestamp for imported.

Constraints and indexes: Index ix_qap_security_scan_results_request_id: request_id | Index ix_qap_security_scan_results_request_type: request_type

qap_suppression_items

SuppressionItem. Individual findings covered by a suppression request.

6 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
suppression_request_id	INTEGER	FK→qap_suppression_requests.id; NULL	Reference to the associated suppression request record.
issue_id	VARCHAR(64)	NULL	Reference to the associated issue record.
severity	VARCHAR(16)	NULL	Severity.
description	TEXT	NULL	Description.
justification	TEXT	NULL	Justification.

qap_suppression_requests

SuppressionRequest. False-positive/suppression approval workflow linked to SAST or DAST work.

25 columns; primary key: id; 7 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
suppression_id	VARCHAR(40)	NULL; UNIQUE; default=_default	Reference to the associated suppression record.
application_name	VARCHAR(150)	NOT NULL	Application name.
scan_type	VARCHAR(16)	NULL	Scan type.
department	VARCHAR(150)	NULL	Department.
application_owner	VARCHAR(150)	NULL	Application owner.
sast_request_id	INTEGER	FK→qap_sast_requests.id; NULL	Reference to the associated sast request record.
dast_request_id	INTEGER	FK→qap_dast_requests.id; NULL	Reference to the associated dast request record.
risk_assessment	TEXT	NULL	Risk assessment.
status	VARCHAR(40)	NULL; default=Draft	Current governed lifecycle state.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
app_owner_decision	VARCHAR(16)	NULL	App owner decision.
app_owner_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated app owner record.
app_owner_decided_at	DATETIME	NULL	Timestamp for app owner decided.
sm_decision	VARCHAR(16)	NULL	Sm decision.
sm_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated sm record.
sm_decided_at	DATETIME	NULL	Timestamp for sm decided.
dept_head_decision	VARCHAR(16)	NULL	Dept head decision.
dept_head_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated dept head record.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
dept_head_decided_at	DATETIME	NULL	Timestamp for dept head decided.
security_decision	VARCHAR(16)	NULL	Security decision.
security_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated security record.
security_decided_at	DATETIME	NULL	Timestamp for security decided.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Unique: suppression_id | Index ix_qap_sup_dept: department

Test repository and execution

qap_test_case_tags

TestCaseTag. Normalized tags attached to a testcase.

3 columns; primary key: id; 1 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
test_case_id	INTEGER	FK→qap_test_cases.id; NOT NULL; INDEX	Reference to the associated test case record.
tag	VARCHAR(80)	NOT NULL; INDEX	Tag.

Constraints and indexes: Unique: test_case_id, tag | Index ix_qap_test_case_tags_tag: tag | Index ix_qap_test_case_tags_test_case_id: test_case_id

qap_test_case_version_steps

TestCaseVersionStep. Ordered execution steps belonging to one exact testcase version.

5 columns; primary key: id; 1 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
version_id	INTEGER	FK→qap_test_case_versions.id; NOT NULL; INDEX	Reference to the associated version record.
step_no	INTEGER	NOT NULL	Step no.
step_text	TEXT	NULL	Step text.
expected_result	TEXT	NULL	Expected result.

Constraints and indexes: Unique: version_id, step_no | Index ix_qap_test_case_version_steps_version_id: version_id

qap_test_case_versions

TestCaseVersion. Immutable governed testcase snapshots, review assignments and version decisions.

29 columns; primary key: id; 8 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
test_case_id	INTEGER	FK→qap_test_cases.id; NOT NULL; INDEX	Reference to the associated test case record.
version_major	INTEGER	NOT NULL; default=1	Version major.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
version_minor	INTEGER	NOT NULL; default=0	Version minor.
status	VARCHAR(30)	NOT NULL; default=Draft	Current governed lifecycle state.
epic_id	VARCHAR(60)	NULL	Reference to the associated epic record.
cr_number	VARCHAR(64)	NULL	Cr number.
feature_id	VARCHAR(60)	NULL	Reference to the associated feature record.
user_story_id	VARCHAR(60)	NULL	Reference to the associated user story record.
test_type	VARCHAR(60)	NULL	Test type.
module_name	VARCHAR(150)	NULL	Module name.
test_scenario	VARCHAR(255)	NULL	Test scenario.
pre_condition	TEXT	NULL	Pre condition.
description	TEXT	NULL	Description.
priority	VARCHAR(16)	NULL	Priority.
author_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated author record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
submitted_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated submitted by record.
submitted_at	DATETIME	NULL	Timestamp for submitted.
submit_note	TEXT	NULL	Submit note.
reviewed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated reviewed by record.
reviewed_at	DATETIME	NULL	Timestamp for reviewed.
review_comments	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
qa_lead_decided_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated qa lead decided by record.
qa_lead_decided_at	DATETIME	NULL	Timestamp for qa lead decided.
qa_lead_decision_comments	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
assigned_reviewer_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assigned reviewer record.
assigned_qa_lead_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assigned qa lead record.
source_version_id	INTEGER	FK→qap_test_case_versions.id; NULL	Reference to the associated source version record.

Constraints and indexes: Unique: test_case_id, version_major, version_minor | Index
ix_qap_test_case_versions_test_case_id: test_case_id

qap_test_cases

TestCase. Stable testcase identity, current pointers and list-compatible mirrors of governed content.

28 columns; primary key: id; 7 foreign keys; 4 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
test_case_key	VARCHAR(60)	NOT NULL; UNIQUE	Test case key.
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL	Reference to the associated project record.
folder_id	INTEGER	FK→qap_test_folders.id; NULL	Reference to the associated folder record.
epic_id	VARCHAR(60)	NULL	Reference to the associated epic record.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
cr_number	VARCHAR(64)	NULL	Cr number.
feature_id	VARCHAR(60)	NULL	Reference to the associated feature record.
user_story_id	VARCHAR(60)	NULL	Reference to the associated user story record.
test_type	VARCHAR(60)	NULL	Test type.
module_name	VARCHAR(150)	NULL	Module name.
test_scenario	VARCHAR(255)	NULL	Test scenario.
pre_condition	TEXT	NULL	Pre condition.
description	TEXT	NULL	Description.
priority	VARCHAR(16)	NULL	Priority.
status	VARCHAR(30)	NULL; default=Draft	Current governed lifecycle state.
version_major	INTEGER	NOT NULL; default=1	Version major.
version_minor	INTEGER	NOT NULL; default=0	Version minor.
current_approved_version_id	INTEGER	FK→qap_test_case_versions.id; NULL	Reference to the associated current approved version record.
current_draft_version_id	INTEGER	FK→qap_test_case_versions.id; NULL	Reference to the associated current draft version record.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NULL; default=now	Last persisted update timestamp.
checked_out_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated checked out by record.
checked_out_at	DATETIME	NULL	Timestamp for checked out.
is_deleted	BOOLEAN	NOT NULL; default=False	Boolean flag indicating deleted.
deleted_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated deleted by record.
deleted_at	DATETIME	NULL	Timestamp for deleted.
deleted_reason	VARCHAR(1000)	NULL	Human-entered rationale retained for workflow/audit traceability.

Constraints and indexes: Unique: test_case_key | Index ix_qap_tc_author_created: created_by_id, created_at | Index ix_qap_tc_proj_del_id: project_id, is_deleted, id | Index ix_qap_tc_proj_deleted_created: project_id, is_deleted, created_at | Index ix_qap_tc_proj_folder_created: project_id, folder_id, created_at

qap_test_cycle_child_links

TestCycleChildRequestLink. Explicit links between a cycle and specialized QA child requests.

5 columns; primary key: id; 1 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
cycle_id	INTEGER	FK→qap_test_cycles.id; NOT NULL; UNIQUE	Reference to the associated cycle record.
child_type	VARCHAR(20)	NOT NULL	Child type.
child_id	INTEGER	NOT NULL; INDEX	Reference to the associated child record.
child_key	VARCHAR(40)	NOT NULL; INDEX	Child key.

Constraints and indexes: Unique: cycle_id | Index ix_qap_test_cycle_child_links_child_id: child_id | Index ix_qap_test_cycle_child_links_child_key: child_key

qap_test_cycle_folder_access

TestCycleFolderAccess. User/department access grants for a restricted cycle folder.

6 columns; primary key: id; 3 foreign keys; 4 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
folder_id	INTEGER	FK→qap_test_cycle_folders.id; NOT NULL; INDEX	Reference to the associated folder record.
department	VARCHAR(150)	NULL	Department.
user_id	INTEGER	FK→qap_users.id; NULL; INDEX	Reference to the associated user record.
granted_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated granted by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Index ix_qap_test_cycle_folder_access_folder_id: folder_id | Index ix_qap_test_cycle_folder_access_user_id: user_id | Index uq_qap_tcfa_folder_department: department (unique) | Index uq_qap_tcfa_folder_user: user_id (unique)

qap_test_cycle_folders

TestCycleFolder. Execution-side cycle folders scoped to a test project.

5 columns; primary key: id; 2 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL; INDEX	Reference to the associated project record.
name	VARCHAR(150)	NOT NULL	Name.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Index ix_qap_test_cycle_folders_project_id: project_id

qap_test_cycles

TestCycle. Execution set, environment/build context, lifecycle status and QA-request linkage.

15 columns; primary key: id; 4 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
cycle_key	VARCHAR(40)	NULL; UNIQUE; default=_default	Cycle key.
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL	Reference to the associated project record.
name	VARCHAR(150)	NOT NULL	Name.
description	TEXT	NULL	Description.
status	VARCHAR(20)	NULL; default=Draft	Current governed lifecycle state.
start_date	DATE	NULL	Start date.
end_date	DATE	NULL	End date.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
cycle_type	VARCHAR(30)	NULL	Cycle type.
environment	VARCHAR(60)	NULL	Environment.
build	VARCHAR(100)	NULL	Build.
owner_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated owner record.
folder_id	INTEGER	FK→qap_test_cycle_folders.id; NULL; INDEX	Reference to the associated folder record.

Constraints and indexes: Unique: cycle_key | Index ix_qap_cyc_proj_status_created: project_id, status, created_at | Index ix_qap_test_cycles_folder_id: folder_id

qap_test_execution_runs

TestExecutionRun. Append-only execution attempts with actual result, evidence context and timestamps.

9 columns; primary key: id; 2 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
execution_id	INTEGER	FK→qap_test_executions.id; NOT NULL	Reference to the associated execution record.
attempt_no	INTEGER	NOT NULL	Attempt no.
status	VARCHAR(20)	NOT NULL	Current governed lifecycle state.
actual_result	TEXT	NULL	Actual result.
test_run_artifacts	VARCHAR(255)	NULL	Test run artifacts.
defect_id	VARCHAR(60)	NULL	Reference to the associated defect record.
executed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated executed by record.
executed_at	DATETIME	NULL; default=now	Timestamp for executed.

Constraints and indexes: Unique: execution_id, attempt_no | Index ix_qap_run_actor_executed: executed_by_id, executed_at

qap_test_executions

TestExecution. One testcase/version slot in a cycle, including assignment and current outcome.

16 columns; primary key: id; 7 foreign keys; 4 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
cycle_id	INTEGER	FK→qap_test_cycles.id; NOT NULL	Reference to the associated cycle record.
test_case_id	INTEGER	FK→qap_test_cases.id; NOT NULL	Reference to the associated test case record.
pinned_version_id	INTEGER	FK→qap_test_case_versions.id; NULL	Reference to the associated pinned version record.
status	VARCHAR(20)	NULL; default=Not Executed	Current governed lifecycle state.
actual_result	TEXT	NULL	Actual result.
test_run_artifacts	VARCHAR(255)	NULL	Test run artifacts.
defect_id	VARCHAR(60)	NULL	Reference to the associated defect record.
assigned_to_id	INTEGER	FK→qap_users.id; NULL; INDEX	Reference to the associated assigned to record.
assigned_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assigned by record.
assigned_at	DATETIME	NULL	Timestamp for assigned.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
executed_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated executed by record.
executed_at	DATETIME	NULL	Timestamp for executed.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
run_version	INTEGER	NOT NULL; default=0	Run version.
added_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated added by record.

Constraints and indexes: Unique: cycle_id, test_case_id | Index ix_qap_te_cyc_asgn_id: cycle_id, assigned_to_id, id | Index ix_qap_te_cyc_id: cycle_id, id | Index ix_qap_te_cyc_stat_id: cycle_id, status, id | Index ix_qap_test_executions_assigned_to_id: assigned_to_id

qap_test_folders

TestFolder. Hierarchical repository folders within a test project.

6 columns; primary key: id; 3 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL; INDEX	Reference to the associated project record.
parent_id	INTEGER	FK→qap_test_folders.id; NULL	Reference to the associated parent record.
name	VARCHAR(150)	NOT NULL	Name.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Index ix_qap_test_folders_project_id: project_id

qap_test_project_members

TestProjectMember. Project membership and project-specific operating role.

6 columns; primary key: id; 3 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL; INDEX	Reference to the associated project record.
user_id	INTEGER	FK→qap_users.id; NOT NULL; INDEX	Reference to the associated user record.
project_role	VARCHAR(30)	NOT NULL; default=Tester	Project role.
added_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated added by record.
added_at	DATETIME	NULL; default=now	Timestamp for added.

Constraints and indexes: Unique: project_id, user_id | Index ix_qap_test_project_members_project_id: project_id | Index ix_qap_test_project_members_user_id: user_id

qap_test_project_view_grants

TestProjectViewGrant. Additional user/department read access to otherwise restricted projects.

6 columns; primary key: id; 3 foreign keys; 4 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
project_id	INTEGER	FK→qap_test_projects.id; NOT NULL; INDEX	Reference to the associated project record.
department	VARCHAR(150)	NULL	Department.
user_id	INTEGER	FK→qap_users.id; NULL; INDEX	Reference to the associated user record.
granted_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated granted by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Index ix_qap_test_project_view_grants_project_id: project_id | Index ix_qap_test_project_view_grants_user_id: user_id | Index uq_qap_tpvq_project_department: department (unique) | Index uq_qap_tpvq_project_user: user_id (unique)

qap_test_projects

TestProject. Test-management access boundary, defaults, lifecycle and application/department ownership.

19 columns; primary key: id; 7 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
project_key	VARCHAR(40)	NULL; UNIQUE; default=_default	Project key.
name	VARCHAR(150)	NOT NULL	Name.
application_master_id	INTEGER	FK→qap_application_master.id; NULL	Reference to the associated application master record.
department	VARCHAR(150)	NULL	Department.
description	TEXT	NULL	Description.
is_active	BOOLEAN	NULL; default=True	Whether the record is active/available for normal use.
owner_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated owner record.
created_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated created by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.
pending_is_active	BOOLEAN	NULL	Pending is active.
pending_requested_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated pending requested by record.
pending_requested_at	DATETIME	NULL	Timestamp for pending requested.
is_archived	BOOLEAN	NULL; default=False	Boolean flag indicating archived.
archived_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated archived by record.
archived_at	DATETIME	NULL	Timestamp for archived.
archived_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
default_reviewer_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated default reviewer record.
default_qa_lead_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated default qa lead record.

Constraints and indexes: Unique: project_key | Index ix_qap_proj_dept: department

qap_test_run_defects

TestRunDefect. Lightweight run-level external defect references used by execution gates.

9 columns; primary key: id; 2 foreign keys; 1 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
run_id	INTEGER	FK→qap_test_execution_runs.id; NOT NULL; INDEX	Reference to the associated run record.
defect_key	VARCHAR(100)	NOT NULL	Defect key.
defect_url	VARCHAR(500)	NULL	Defect url.
title	VARCHAR(255)	NULL	Title.
defect_status	VARCHAR(40)	NULL	State/disposition value for defect status.
notes	TEXT	NULL	Notes.
linked_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated linked by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: run_id, defect_key | Index ix_qap_test_run_defects_run_id: run_id

qap_test_steps

TestStep. Legacy/current testcase steps associated with the stable testcase record.

5 columns; primary key: id; 1 foreign keys.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
test_case_id	INTEGER	FK→qap_test_cases.id; NOT NULL	Reference to the associated test case record.
step_no	INTEGER	NOT NULL	Step no.
step_text	TEXT	NULL	Step text.
expected_result	TEXT	NULL	Expected result.

Defect management

qap_defect_case_links

DefectTestCaseLink. Many-to-many traceability between governed defects and testcases.

3 columns; primary key: id; 2 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
defect_id	INTEGER	FK→qap_defects.id; NOT NULL; INDEX	Reference to the associated defect record.
test_case_id	INTEGER	FK→qap_test_cases.id; NOT NULL; INDEX	Reference to the associated test case record.

Constraints and indexes: Unique: defect_id, test_case_id | Index ix_qap_defect_case_links_defect_id: defect_id | Index ix_qap_defect_case_links_test_case_id: test_case_id

gap_defect_execution_links

DefectExecutionLink. Many-to-many traceability between defects and cycle execution records/runs.

5 columns; primary key: id; 3 foreign keys; 2 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
defect_id	INTEGER	FK→qap_defects.id; NOT NULL; INDEX	Reference to the associated defect record.
execution_id	INTEGER	FK→qap_test_executions.id; NOT NULL; INDEX	Reference to the associated execution record.
linked_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated linked by record.
created_at	DATETIME	NULL; default=now	Server-side creation timestamp.

Constraints and indexes: Unique: defect_id, execution_id | Index ix_qap_defect_execution_links_defect_id: defect_id | Index ix_qap_defect_execution_links_execution_id: execution_id

qap_defects

Defect. Governed defect identity, triage, assignment, resolution, retest and closure evidence.

58 columns; primary key: id; 9 foreign keys; 12 explicit indexes.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
id	INTEGER	PK; NOT NULL; server default	Surrogate numeric primary key.
defect_key	VARCHAR(40)	NOT NULL; UNIQUE; INDEX	Defect key.
title	VARCHAR(255)	NOT NULL	Title.
description	TEXT	NOT NULL	Description.
status	VARCHAR(20)	NOT NULL; INDEX; default=New	Current governed lifecycle state.
qa_request_id	INTEGER	FK→qap_requests.id; NOT NULL; INDEX	Reference to the associated qa request record.
cycle_id	INTEGER	FK→qap_test_cycles.id; NULL; INDEX	Reference to the associated cycle record.
primary_test_case_id	INTEGER	FK→qap_test_cases.id; NULL; INDEX	Reference to the associated primary test case record.
execution_id	INTEGER	FK→qap_test_executions.id; NULL; INDEX	Reference to the associated execution record.
application_name	VARCHAR(150)	NOT NULL	Application name.
module_feature	VARCHAR(150)	NOT NULL	Module feature.
environment	VARCHAR(60)	NOT NULL	Environment.
severity	VARCHAR(20)	NOT NULL; INDEX	Severity.
priority	VARCHAR(20)	NOT NULL; INDEX	Priority.
steps_to_reproduce	TEXT	NOT NULL	Steps to reproduce.
expected_result	TEXT	NOT NULL	Expected result.
actual_result	TEXT	NOT NULL	Actual result.
reporter_id	INTEGER	FK→qap_users.id; NOT NULL; INDEX	Reference to the associated reporter record.
reported_at	DATETIME	NOT NULL; default=now	Timestamp for reported.
assignee_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assignee record.
assigned_team	VARCHAR(150)	NULL	Assigned team.
assigned_by_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated assigned by record.
assigned_at	DATETIME	NULL	Timestamp for assigned.

Column	SQLAlchemy type	Rules / reference	Implementation meaning
assignment_remarks	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
retest_tester_id	INTEGER	FK→qap_users.id; NULL	Reference to the associated retest tester record.
device_details	VARCHAR(255)	NULL	Device details.
build_version	VARCHAR(100)	NULL	Build version.
api_endpoint	VARCHAR(500)	NULL	Api endpoint.
request_response_details	TEXT	NULL	Request response details.
log_details	TEXT	NULL	Log details.
related_cr_number	VARCHAR(64)	NULL	Related cr number.
external_defect_id	VARCHAR(100)	NULL	Reference to the associated external defect record.
remarks	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
labels	VARCHAR(500)	NULL	Labels.
resolution_type	VARCHAR(60)	NULL	Resolution type.
resolution_summary	TEXT	NULL	Resolution summary.
root_cause	TEXT	NULL	Root cause.
fix_details	TEXT	NULL	Fix details.
fixed_build_version	VARCHAR(100)	NULL	Fixed build version.
resolved_at	DATETIME	NULL	Timestamp for resolved.
retest_result	VARCHAR(20)	NULL	Retest result.
retest_at	DATETIME	NULL	Timestamp for retest.
tested_build_version	VARCHAR(100)	NULL	Tested build version.
retest_actual_result	TEXT	NULL	Retest actual result.
retest_remarks	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
reopen_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
reopen_count	INTEGER	NOT NULL; default=0	Stored aggregate count for reopen.
deferral_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
deferral_approved_by	VARCHAR(150)	NULL	Deferral approved by.
target_release	VARCHAR(100)	NULL	Target release.
expected_resolution_date	DATE	NULL	Expected resolution date.
rejection_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
duplicate_of_id	INTEGER	FK→qap_defects.id; NULL	Reference to the associated duplicate of record.
not_a_defect_reason	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
closure_remarks	TEXT	NULL	Human-entered rationale retained for workflow/audit traceability.
closed_at	DATETIME	NULL	Timestamp for closed.
created_at	DATETIME	NOT NULL; default=now	Server-side creation timestamp.
updated_at	DATETIME	NOT NULL; default=now	Last persisted update timestamp.

Constraints and indexes: Index ix_qap_def_reporter_at: reporter_id, reported_at | Index ix_qap_def_retester_at: retest_tester_id, retest_at | Index ix_qap_defects_assignee_upd: assignee_id, status, updated_at | Index ix_qap_defects_cycle_id: cycle_id | Index ix_qap_defects_defect_key: defect_key (unique) | Index ix_qap_defects_execution_id: execution_id | Index ix_qap_defects_primary_test_case_id: primary_test_case_id | Index ix_qap_defects_priority: priority | Index ix_qap_defects_qa_request_id: qa_request_id | Index ix_qap_defects_reporter_id: reporter_id | Index ix_qap_defects_severity: severity | Index ix_qap_defects_status: status

Appendix D. Complete REST API endpoint catalog

The current codebase defines 389 route registrations across 22 router modules (388 OpenAPI-visible and 1 compatibility alias). This catalog is generated from FastAPI decorators and includes every registered method/path pair. Authorization labels summarize declared dependencies; inline ownership, department, assignment and workflow-state checks remain authoritative.

Router module	Endpoint registrations	Primary prefix / surface
applications	9	/api/application-names
approvals	8	/api/approvals
audit	4	/api/audit
auth	12	/api/auth
checklist_config	7	/api/checklist-config
dashboard	11	/api/dashboard
defects	13	/api/defects
departments	4	/api/departments
export	2	/api/export
functional	36	/api/functional-requests
jobs	2	/api/jobs
pending_approvals	2	/api/pending-approvals
performance	34	/api/performance-requests
qa_requests	24	/api/qa-requests
reports	12	/api/reports
sast_dast	72	/api/dast-requests, /api/sast-requests
signoff	15	/api/signoffs
suppression	16	/api/suppressions
test_execution	40	/api/test-execution
test_projects	16	/api/test-projects
test_reports	5	/api/test-reports
test_repository	45	/api/test-repository

Interpretation Public / route-specific means the decorator did not expose a standard current-user dependency in its function signature; consult the implementation before treating the route as anonymous. Hidden compatibility aliases remain callable but are excluded from generated OpenAPI documentation.

applications router

9 method/path registrations. Source: backend/app/routers/applications.py

Method	Path	Operation	Declared access	Response
GET	/api/application-names	Approved names only -- this is what feeds the QA Request wizard's Application Name dropdown (see frontend QARequests/steps/DetailsStep.tsx).	Authenticated	List[schemas.ApplicationMasterOut]
POST	/api/application-names/bulk-seed	Admin-only bulk import: seeds ApplicationMaster directly from an xlsx of known-good application names.	ADMIN	schemas.ApplicationSeedResult
GET	/api/application-names/bulk-seed-template	Minimal xlsx template for bulk_seed_application_names below, built on the fly with openpyxl rather than a bundled static asset (contrast wit	ADMIN	—
GET	/api/application-names/pending	SM/Admin section listing every name that has cleared Application Owner and is now awaiting SM, the second and final tier -- mainly useful fo	SM, ADMIN	List[schemas.ApplicationMasterOut]
GET	/api/application-names/pending-app-owner	Application Owner/Admin section listing every name still awaiting the FIRST tier of the two-tier approval chain -- mirrors list_pending_appl	APPLICATION_OWNER, ADMIN	List[schemas.ApplicationMasterOut]
POST	/api/application-names/{app_id}/app-owner-decision	Single-tier Application Name approval (2026-08 v2).	APPLICATION_OWNER	schemas.ApplicationMasterOut
POST	/api/application-names/{app_id}/decision	LEGACY-ONLY as of 2026-08 v2 (see decide_app_owner_name's own docstring -- reported directly: "only application owner approval required, no	SM	schemas.ApplicationMasterOut
PATCH	/api/application-names/{app_id}/department	Assign an existing application master to an active system department.	ADMIN	schemas.ApplicationMasterOut
PATCH	/api/application-names/{app_id}/name	Renaming the canonical Application Name -- reported directly: "edit option for renaming the application name" on the Existing Application Ma	ADMIN	schemas.ApplicationMasterOut

approvals router

8 method/path registrations. Source: backend/app/routers/approvals.py

Method	Path	Operation	Declared access	Response
GET	/api/approvals	Module 7: cross-entity approval/audit feed (QA_REQUEST, TEST_CASE, SAST_DAST, SUPPRESSION, SIGNOFF).	Authenticated	List[schemas.ApprovalActionOut]
GET	/api/approvals/assignment-history	Return normalized assignee tenures for management/audit reporting.	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA, DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	List[schemas.AssignmentHistoryOut]
GET	/api/approvals/comments/{comment_id}/attachments	List comment attachments	Authenticated	List[schemas.RequestDocumentOut]

Method	Path	Operation	Declared access	Response
GET	/api/approvals/comments/{comment_id}/attachments/{document_id}/download	Download comment attachment	Authenticated	—
GET	/api/approvals/history	SRS 7.2 pagination rollout -- backs modules/governance/Approvals.tsx's "Approval Workflow Log," the one screen that genuinely paginates thro	Authenticated	pagination.Page[schemas.ApprovalActionOut]
GET	/api/approvals/pending-mine	My recent actions	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/approvals/{entity_type}/{entity_id}/comments	Append a Jira-style standalone comment to any supported module's existing audit stream.	Authenticated	schemas.ApprovalActionOut
POST	/api/approvals/{entity_type}/{entity_id}/rich-comments	Post formatted comment text plus images pasted or selected in the shared Jira-style editor.	Authenticated	schemas.ApprovalActionOut

audit router

4 method/path registrations. Source: backend/app/routers/audit.py

Method	Path	Operation	Declared access	Response
GET	/api/audit	SRS 7.2 pagination rollout -- this endpoint already did real SQL "OFFSET"/"LIMIT" pagination before this change (unlike most of the endpoint	*AUDIT_ROLES	pagination.Page[schemas.AuditLogOut]
GET	/api/audit/export	Export audit logs	*AUDIT_ROLES	—
GET	/api/audit/summary	Same filters as 'list_audit_logs', so the summary strip always reflects the currently active filter set (matching what the old single-respon	*AUDIT_ROLES	schemas.AuditSummary
GET	/api/audit/{audit_id}	Get audit log	*AUDIT_ROLES	schemas.AuditLogOut

auth router

12 method/path registrations. Source: backend/app/routers/auth.py

Method	Path	Operation	Declared access	Response
GET	/api/auth/local-admin/users	Every user mapped to the local admin's own department (any status, so a previously-disabled account can be re-activated too), excluding thei	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM, CHIEF_MANAGER_QA, AGM_QA	list[schemas.UserOut]
PATCH	/api/auth/local-admin/users/{user_id}	Update local admin user	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM, CHIEF_MANAGER_QA, AGM_QA	schemas.UserOut
POST	/api/auth/login	Login	Public / route-specific	schemas.Token
POST	/api/auth/logout	Logout	Authenticated	—
GET	/api/auth/me	Me	Authenticated	schemas.UserOut
PATCH	/api/auth/me	One-time LDAP self-service selection of exactly one primary department.	Authenticated	schemas.UserOut
GET	/api/auth/users	Active users only -- used throughout the app for pickers (assign tester, etc.).	Authenticated	list[schemas.UserOut]
POST	/api/auth/users	Admin section (Module 9): user mapping = department(s) + one or more roles (access types).	ADMIN	schemas.UserOut
GET	/api/auth/users/all	Admin section (Module 9): full user directory, including disabled accounts.	ADMIN	pagination.Page[schemas.UserOut]

Method	Path	Operation	Declared access	Response
GET	/api/auth/users/summary	User summary	ADMIN	schemas.UserSummaryOut
PATCH	/api/auth/users/{user_id}	Admin section (Module 9): reassign role(s)/department(s), change login type, activate/deactivate, edit profile fields.	ADMIN	schemas.UserOut
POST	/api/auth/users/{user_id}/reset-password	Admin section (Module 9): set/reset a Standard account's local password.	ADMIN	schemas.UserOut

checklist_config router

7 method/path registrations. Source: backend/app/routers/checklist_config.py

Method	Path	Operation	Declared access	Response
GET	/api/checklist-config/modules	Just the module keys + their "detail" column label -- lets the Admin page and the QA Request wizard build their tabs/labels without hardcodi	Authenticated	—
GET	/api/checklist-config/{module}	Active items only, ordered -- this is what the QA Request wizard's Functional/SAST/DAST/Performance steps render as the self-declaration che	Authenticated	List[schemas.ChecklistTemplateItemOut]
POST	/api/checklist-config/{module}	Create item	ADMIN	schemas.ChecklistTemplateItemOut
GET	/api/checklist-config/{module}/all	Admin management view -- includes disabled items too, so they can be re-enabled instead of only ever being re-typed from scratch.	ADMIN	List[schemas.ChecklistTemplateItemOut]
POST	/api/checklist-config/{module}/restore-defaults	Wipes every configured item for this module and reseeds from the shipped defaults (constants.DEFAULT_*_CHECKLIST_ITEMS) -- an escape hatch b	ADMIN	List[schemas.ChecklistTemplateItemOut]
DELETE	/api/checklist-config/{module}/{item_id}	Hard delete -- safe to do even though older, already-raised requests keep referencing this same item text, because their own checklist rows	ADMIN	—
PATCH	/api/checklist-config/{module}/{item_id}	Renaming, retexting, toggling Mandatory/Active, or moving an item are all the same "edit this row" action -- exactly what "whatever I mentio	ADMIN	schemas.ChecklistTemplateItemOut

dashboard router

11 method/path registrations. Source: backend/app/routers/dashboard.py

Method	Path	Operation	Declared access	Response
GET	/api/dashboard/3w	'Know What Is Pending, Where It Is Pending, and Since When' -- section 4.9.8.	Authenticated	—
GET	/api/dashboard/3w/{project_id}	Drill-down: selecting a Project ID shows its full lifecycle + audit trail.	Authenticated	—
GET	/api/dashboard/attention/{metric}	Record-level reconciliation for the four Command Centre cards.	Authenticated	—
GET	/api/dashboard/project-wise	Project wise	Authenticated	—
GET	/api/dashboard/qa-contribution-export	Excel evidence pack for the visible management contribution scope.	Authenticated	—
GET	/api/dashboard/qa-tester-contribution/{tester_id}	Drill-down evidence behind one tester's contribution metrics.	Authenticated	—

Method	Path	Operation	Declared access	Response
GET	/api/dashboard/qa-tester-workload	QA-team-only capacity view.	Authenticated	—
GET	/api/dashboard/security/dast	Security dast	Authenticated	—
GET	/api/dashboard/security/sast	Security sast	Authenticated	—
GET	/api/dashboard/summary	DSH-001..004 -- consolidates the handful of derived numbers CommandCentre used to compute in the browser (see Dashboard.tsx's own comment on	Authenticated	—
GET	/api/dashboard/suppression	Suppression dashboard	Authenticated	—

defects router

13 method/path registrations. Source: backend/app/routers/defects.py

Method	Path	Operation	Declared access	Response
GET	/api/defects	List defects	Authenticated	pagination.Page[schemas.DefectListOut]
POST	/api/defects	Create defect	Authenticated	schemas.DefectOut
GET	/api/defects/by-key/{defect_key}	SRS 7.2 pagination rollout -- Defects.tsx's own '?open=<defect_key>' deep-link (used both for Global Search and for the create-defect flow's	Authenticated	schemas.DefectOut
GET	/api/defects/dashboard	Defect dashboard	Authenticated	schemas.DefectDashboardOut
GET	/api/defects/export-xlsx	Export the governed defect register with workflow and traceability fields.	Authenticated	—
GET	/api/defects/{defect_id}	Get defect	Authenticated	schemas.DefectOut
PATCH	/api/defects/{defect_id}	Update defect	Authenticated	schemas.DefectOut
GET	/api/defects/{defect_id}/attachments	List attachments	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/defects/{defect_id}/attachments	Upload attachments	Authenticated	List[schemas.RequestDocumentOut]
GET	/api/defects/{defect_id}/attachments/{document_id}/download	Download attachment	Authenticated	—
POST	/api/defects/{defect_id}/link-execution	Link defect execution	Authenticated	schemas.DefectOut
POST	/api/defects/{defect_id}/reassign	Reassign defect	Authenticated	schemas.DefectOut
POST	/api/defects/{defect_id}/transition	Transition defect	Authenticated	schemas.DefectOut

departments router

4 method/path registrations. Source: backend/app/routers/departments.py

Method	Path	Operation	Declared access	Response
GET	/api/departments	Active departments only -- used by every department picker in the app (Admin > Create User / assign department, etc).	Authenticated	List[schemas.DepartmentOut]
POST	/api/departments	Create department	ADMIN	schemas.DepartmentOut
GET	/api/departments/all	Admin section: full list including deactivated departments, for management.	ADMIN	List[schemas.DepartmentOut]
PATCH	/api/departments/{dept_id}	Rename a department and/or activate/deactivate it.	ADMIN	schemas.DepartmentOut

export router

2 method/path registrations. Source: backend/app/routers/export.py

Method	Path	Operation	Declared access	Response
GET	/api/export/reports	Available reports	Authenticated	—
GET	/api/export/{report_key}	Module 11: Download & Export Centre.	Authenticated	—

functional router

36 method/path registrations. Source: backend/app/routers/functional.py

Method	Path	Operation	Declared access	Response
GET	/api/functional-requests	List functional	Authenticated	pagination.Page[schemas.FunctionalListOut]
POST	/api/functional-requests	Create functional	Authenticated	schemas.FunctionalOutput
GET	/api/functional-requests/{req_id}	Get functional	Authenticated	schemas.FunctionalOutput
PUT	/api/functional-requests/{req_id}	Lets the requester (or SM/Department Head of the requester's own department) fix up a Functional Testing Request's details after it's been r	Authenticated	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/assign-tester	2026-08 -- reported directly: "once assigned there are no other option to reassign the tester or modify the tester." Previously only ever ca	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA, QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/begin-planning	Begin planning	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.FunctionalOutput
GET	/api/functional-requests/{req_id}/checklist	Get checklist	Authenticated	List[schemas.ChecklistItemOut]
GET	/api/functional-requests/{req_id}/checklist/documents	Batched counterpart to list_functional_checklist_documents below -- see ChecklistItemDocumentOut for why this exists.	Authenticated	List[schemas.ChecklistItemDocumentOut]
PUT	/api/functional-requests/{req_id}/checklist/{item_id}	Update checklist item	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.ChecklistItemOut
GET	/api/functional-requests/{req_id}/checklist/{item_id}/documents	List functional checklist documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/functional-requests/{req_id}/checklist/{item_id}/documents	Upload functional checklist documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/functional-requests/{req_id}/checklist/{item_id}/documents/{doc_id}	Delete functional checklist document	Authenticated	—
GET	/api/functional-requests/{req_id}/checklist/{item_id}/documents/{doc_id}/download	Download functional checklist document	Authenticated	—
POST	/api/functional-requests/{req_id}/complete-qa	Marks QA activity complete -- reachable directly from execution (no issues found) or after the defect/retest cycle.	QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/confirm-signoff	Confirms the QA Clearance certificate (optionally linking a Module 8 QASignOff record created via /api/signoffs) and hands the request to th	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/department-head-decision	Department Head reviews the request and assigns a COE - Quality Assurance QA Lead.	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	schemas.FunctionalOutput
GET	/api/functional-requests/{req_id}/documents	List functional documents	Authenticated	List[schemas.RequestDocumentOut]

Method	Path	Operation	Declared access	Response
POST	/api/functional-requests/{req_id}/documents	Upload functional documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/functional-requests/{req_id}/documents/{doc_id}	Delete functional document	Authenticated	—
GET	/api/functional-requests/{req_id}/documents/{doc_id}/download	Download functional document	Authenticated	—
GET	/api/functional-requests/{req_id}/eligible-test-cycles	Eligible test cycles	QA_LEAD, QA_ENGINEER	List[schemas.EligibleTestCycleOut]
GET	/api/functional-requests/{req_id}/export	Every field on this Functional QA request (including the ones delegated from its parent QA Request gateway), its readiness checklist, and it	Authenticated	—
GET	/api/functional-requests/{req_id}/history	Request history	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/functional-requests/{req_id}/mark-waiting-for-fix	Mark waiting for fix	QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/raise-defect	Raise defect	QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/readiness-decision	Readiness decision	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/request-signoff	Request signoff	QA_LEAD, QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/requester-decision	Requester decision	REQUESTER, BUSINESS_ANALYST, APPLICATION_OWNER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/resubmit	Re-submits a request returned by SM, by the Department Head, or by the QA Lead -- or reopens one rejected by SM.	REQUESTER, BUSINESS_ANALYST	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/sm-decision	Checkpoint between the requester's submission and Department Head approval.	SM	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/start-execution	Start execution	QA_LEAD, QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/start-readiness-verification	Start readiness verification	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/start-retesting	Start retesting	QA_LEAD, QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/start-test-design	Start test design	QA_ENGINEER	schemas.FunctionalOutput
POST	/api/functional-requests/{req_id}/submit	Submit request	REQUESTER, BUSINESS_ANALYST	schemas.FunctionalOutput
DELETE	/api/functional-requests/{req_id}/test-cycles/{cycle_id}	Unlink test cycle	QA_LEAD, QA_ENGINEER	schemas.FunctionalOutput

jobs router

2 method/path registrations. Source: backend/app/routers/jobs.py

Method	Path	Operation	Declared access	Response
GET	/api/jobs/{job_id}	Get job	Authenticated	—
GET	/api/jobs/{job_id}/download	Download job artifact	Authenticated	—

pending_approvals router

2 method/path registrations. Source: backend/app/routers/pending_approvals.py

Method	Path	Operation	Declared access	Response
GET	/api/pending-approvals	Everything genuinely awaiting the logged-in user's own decision right now, across every approval checkpoint in the app -- see this module's	Authenticated	schemas.PendingApprovalPage
GET	/api/pending-approvals/count	Fast login summary without materializing the detailed approval feed.	Authenticated	schemas.PendingApprovalCount

performance router

34 method/path registrations. Source: backend/app/routers/performance.py

Method	Path	Operation	Declared access	Response
GET	/api/performance-requests	List performance	Authenticated	pagination.Page[schemas.PerformanceListOut]
POST	/api/performance-requests	Create performance	Authenticated	schemas.PerformanceOut
GET	/api/performance-requests/{req_id}	Get performance	Authenticated	schemas.PerformanceOut
PUT	/api/performance-requests/{req_id}	Update performance	Authenticated	schemas.PerformanceOut
GET	/api/performance-requests/{req_id}/checklist	Get checklist	Authenticated	List[schemas.PerformanceChecklistItemOut]
GET	/api/performance-requests/{req_id}/checklist/documents	Batched counterpart to list_performance_checklist_documents below -- see ChecklistItemDocumentOut for why this exists.	Authenticated	List[schemas.ChecklistItemDocumentOut]
PUT	/api/performance-requests/{req_id}/checklist/{item_id}	Update checklist item	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceChecklistItemOut
GET	/api/performance-requests/{req_id}/checklist/{item_id}/documents	List performance checklist documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/performance-requests/{req_id}/checklist/{item_id}/documents	Upload performance checklist documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/performance-requests/{req_id}/checklist/{item_id}/documents/{doc_id}	Delete performance checklist document	Authenticated	—
GET	/api/performance-requests/{req_id}/checklist/{item_id}/documents/{doc_id}/download	Download performance checklist document	Authenticated	—
POST	/api/performance-requests/{req_id}/complete-baseline	Complete baseline	QA_LEAD, QA_ENGINEER, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceOut
POST	/api/performance-requests/{req_id}/complete-defect-fix-retest	Loops back to Load Test Execution for a re-run once the fix is in.	QA_LEAD, QA_ENGINEER, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceOut
POST	/api/performance-requests/{req_id}/complete-environment-setup	Complete environment setup	QA_LEAD, QA_ENGINEER, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceOut
POST	/api/performance-requests/{req_id}/complete-feasibility	Complete feasibility	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceOut
POST	/api/performance-requests/{req_id}/complete-load-test	Complete load test	QA_LEAD, QA_ENGINEER, CHIEF_MANAGER_QA, AGM_QA	schemas.PerformanceOut
POST	/api/performance-requests/{req_id}/complete-planning	2026-08 -- reported directly: "once assigned there are no other option to reassign the tester or modify the tester." Previously only ever ca	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA, QA_ENGINEER	schemas.PerformanceOut

Method	Path	Operation	Declared access	Response
POST	/api/performance-requests/{req_id}/complete-report	Complete report	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/complete-script-development	Complete script development	QA_LEAD, QA_ENGINEER, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/department-head-decision	Approval requires assignment to an active COE - Quality Assurance QA Lead.	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	schemas.Performance Out
GET	/api/performance-requests/{req_id}/documents	List performance documents	Authenticated	List[schemas.Request DocumentOut]
POST	/api/performance-requests/{req_id}/documents	Upload performance documents	Authenticated	List[schemas.Request DocumentOut]
DELETE	/api/performance-requests/{req_id}/documents/{doc_id}	Delete performance document	Authenticated	—
GET	/api/performance-requests/{req_id}/documents/{doc_id}/download	Download performance document	Authenticated	—
GET	/api/performance-requests/{req_id}/export	Every field on this Performance Testing request (including the ones delegated from its parent QA Request gateway), its readiness checklist,	Authenticated	—
GET	/api/performance-requests/{req_id}/history	Request history	Authenticated	List[schemas.Approval ActionOut]
POST	/api/performance-requests/{req_id}/readiness-decision	Passed requires every item the requester self-declared ready on "L1: Pre-Testing Readiness Checklist" (Annexure VIII) to be QA-verified (is_	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/requester-decision	Requester decision	Authenticated	schemas.Performance Out
POST	/api/performance-requests/{req_id}/resubmit	Re-submits a request returned by SM, by the Department Head, or after a failed Readiness review -- or reopens one rejected by SM.	Authenticated	schemas.Performance Out
POST	/api/performance-requests/{req_id}/result-analysis-decision	Result analysis decision	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/sign-off	Sign off	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/sm-decision	Sm decision	SM	schemas.Performance Out
POST	/api/performance-requests/{req_id}/start-readiness	Start readiness	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.Performance Out
POST	/api/performance-requests/{req_id}/submit	Submit performance	Authenticated	schemas.Performance Out

qa_requests router

24 method/path registrations. Source: backend/app/routers/qa_requests.py

Method	Path	Operation	Declared access	Response
GET	/api/qa-requests	SRS 7.2 (PAG-001..009) -- server-side paginated, database-filtered list.	Authenticated	pagination.Page[sche mas.QARequestListOu t]
POST	/api/qa-requests	Creates the gateway request in Draft ONLY -- no linked child request (Functional/SAST/DAST/Performance) is created yet, even if SAST/DAST/et	REQUESTER, BUSINESS_ANALYST	schemas.QARequestO ut
GET	/api/qa-requests/bug-fix-source-options	Recent completed Functional requests eligible as a Bug Fix source.	Authenticated	—
POST	/api/qa-requests/{qa_request_id}/child-delegations/{target_type}/{target_id}	Assign child for input	Authenticated	schemas.QARequestD elegationOut

Method	Path	Operation	Declared access	Response
POST	/api/qa-requests/{qa_request_id}/child-delegations/{target_type}/{target_id}/recall	Recall child delegation	Authenticated	schemas.QARequestDelegationOut
POST	/api/qa-requests/{qa_request_id}/child-delegations/{target_type}/{target_id}/return	Return child delegation	Authenticated	schemas.QARequestDelegationOut
GET	/api/qa-requests/{req_id}	Get request	Authenticated	schemas.QARequestOut
PUT	/api/qa-requests/{req_id}	Edit request	Authenticated	schemas.QARequestOut
POST	/api/qa-requests/{req_id}/cancel	Cancel request	Authenticated	schemas.QARequestOut
GET	/api/qa-requests/{req_id}/checklist-evidence/documents	Batched counterpart to list_draft_checklist_evidence below -- the QA Request wizard renders one ChecklistEvidencePicker per readiness checkl	Authenticated	List[schemas.DraftChecklistEvidenceOut]
GET	/api/qa-requests/{req_id}/checklist-evidence/{kind}/{item_index}/documents	List draft checklist evidence	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/qa-requests/{req_id}/checklist-evidence/{kind}/{item_index}/documents	Upload draft checklist evidence	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/qa-requests/{req_id}/checklist-evidence/{kind}/{item_index}/documents/{doc_id}	Delete draft checklist evidence	Authenticated	—
GET	/api/qa-requests/{req_id}/checklist-evidence/{kind}/{item_index}/documents/{doc_id}/download	Download draft checklist evidence	Authenticated	—
POST	/api/qa-requests/{req_id}/delegations	Assign for input	Authenticated	schemas.QARequestOut
POST	/api/qa-requests/{req_id}/delegations/recall	Recall delegated request	Authenticated	schemas.QARequestOut
POST	/api/qa-requests/{req_id}/delegations/return	Return delegated request	Authenticated	schemas.QARequestOut
GET	/api/qa-requests/{req_id}/documents	List documents	Authenticated	List[schemas.QARequestDocumentOut]
POST	/api/qa-requests/{req_id}/documents	Accepts one or more files (multipart/form-data, field name 'files') and stores them under UPLOAD_ROOT/{request_id}/, named to avoid collision	Authenticated	List[schemas.QARequestDocumentOut]
DELETE	/api/qa-requests/{req_id}/documents/{doc_id}	Delete document	Authenticated	—
GET	/api/qa-requests/{req_id}/documents/{doc_id}/download	Download document	Authenticated	—
GET	/api/qa-requests/{req_id}/export	Every field on this gateway record, plus the full Gateway Actions approval history (who submitted/raised/cancelled it and when), as one down	Authenticated	—
GET	/api/qa-requests/{req_id}/history	Request history	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/qa-requests/{req_id}/submit	Raises the request: creates whichever linked child request(s) the selected request_types call for -- this is the only place they're ever cre	REQUESTER, BUSINESS_ANALYST	schemas.QARequestOut

reports router

12 method/path registrations. Source: backend/app/routers/reports.py

Method	Path	Operation	Declared access	Response
GET	/api/reports/application-quality-scorecard	Cross-module position by application, restricted through visible gateway IDs.	Authenticated	—
GET	/api/reports/audit-evidence	Audit evidence	Authenticated	—

Method	Path	Operation	Declared access	Response
GET	/api/reports/dast-scan	Dast scan report	Authenticated	—
GET	/api/reports/defect-retest-register	Defect retest register	Authenticated	—
GET	/api/reports/performance-testing	Performance testing report	Authenticated	—
GET	/api/reports/qa-request-summary	One row per QA Request (the intake gateway -- see constants.GatewayStatus for its own Draft/Submitted/Raised/Cancelled status).	Authenticated	—
GET	/api/reports/qa-signoff-register	Qa signoff register	Authenticated	—
GET	/api/reports/sast-scan	Sast scan report	Authenticated	—
GET	/api/reports/severity-distribution	Severity distribution	Authenticated	—
GET	/api/reports/suppression-register	Suppression register	Authenticated	—
GET	/api/reports/test-cycle-summary	One compact row per visible cycle; avoids exporting every execution attempt.	Authenticated	—
GET	/api/reports/testcase-approval-summary	Testcase approval summary	Authenticated	—

sast_dast router

72 method/path registrations. Source: backend/app/routers/sast_dast.py

Method	Path	Operation	Declared access	Response
GET	/api/dast-requests	List dast	Authenticated	pagination.Page[schemas.DASTListOut]
POST	/api/dast-requests	Create dast	Authenticated	schemas.DASTOut
GET	/api/dast-requests/{req_id}	Get dast	Authenticated	schemas.DASTOut
PUT	/api/dast-requests/{req_id}	Update dast	Authenticated	schemas.DASTOut
POST	/api/dast-requests/{req_id}/assign-security-analyst	Dast assign security analyst	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA, SECURITY_ANALYST	schemas.DASTOut
POST	/api/dast-requests/{req_id}/assign-to-requester	Dast assign to requester	SECURITY_ANALYST	schemas.DASTOut
GET	/api/dast-requests/{req_id}/checklist	Get dast checklist	Authenticated	List[schemas.ChecklistItemOut]
GET	/api/dast-requests/{req_id}/checklist/documents	Batched counterpart to list_dast_checklist_documents below -- see ChecklistItemDocumentOut for why this exists.	Authenticated	List[schemas.ChecklistItemDocumentOut]
PUT	/api/dast-requests/{req_id}/checklist/{item_id}	Update dast checklist item	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.ChecklistItemOut
GET	/api/dast-requests/{req_id}/checklist/{item_id}/documents	List dast checklist documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/dast-requests/{req_id}/checklist/{item_id}/documents	Upload dast checklist documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/dast-requests/{req_id}/checklist/{item_id}/documents/{doc_id}	Delete dast checklist document	Authenticated	—
GET	/api/dast-requests/{req_id}/checklist/{item_id}/documents/{doc_id}/download	Download dast checklist document	Authenticated	—
POST	/api/dast-requests/{req_id}/close	Dast close request	SECURITY_ANALYST	schemas.DASTOut
POST	/api/dast-requests/{req_id}/department-head-decision	Dast department head decision	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	schemas.DASTOut
GET	/api/dast-requests/{req_id}/documents	List dast documents	Authenticated	List[schemas.RequestDocumentOut]

Method	Path	Operation	Declared access	Response
POST	/api/dast-requests/{req_id}/documents	Upload dast documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/dast-requests/{req_id}/documents/{doc_id}	Delete dast document	Authenticated	—
GET	/api/dast-requests/{req_id}/documents/{doc_id}/download	Download dast document	Authenticated	—
GET	/api/dast-requests/{req_id}/export	Every field on this DAST request (including the ones delegated from its parent QA Request gateway), its scan targets, its findings, and its	Authenticated	—
POST	/api/dast-requests/{req_id}/findings/{finding_id}/resolve	Resolve dast finding	SECURITY_ANALYST	schemas.DASTFindingOut
GET	/api/dast-requests/{req_id}/history	Dast history	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/dast-requests/{req_id}/mark-fixed	Dast mark fixed	Authenticated	schemas.DASTOut
POST	/api/dast-requests/{req_id}/mark-report-ready	Dast mark report ready	SECURITY_ANALYST	schemas.DASTOut
POST	/api/dast-requests/{req_id}/mark-scan-complete	Dast mark scan complete	SECURITY_ANALYST	schemas.DASTOut
POST	/api/dast-requests/{req_id}/readiness-decision	Dast readiness decision	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.DASTOut
POST	/api/dast-requests/{req_id}/rescan	Dast rescan	SECURITY_ANALYST	schemas.DASTScanStartOut
POST	/api/dast-requests/{req_id}/resubmit	Resubmit dast	Authenticated	schemas.DASTOut
GET	/api/dast-requests/{req_id}/scan-results	Dast scan results	Authenticated	List[schemas.SecurityScanResultOut]
GET	/api/dast-requests/{req_id}/scan-summary	Dast scan summary	Authenticated	schemas.SecurityScanSummaryOut
POST	/api/dast-requests/{req_id}/sm-decision	Dast sm decision	SM	schemas.DASTOut
POST	/api/dast-requests/{req_id}/start-configuration	Dast start configuration	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.DASTOut
POST	/api/dast-requests/{req_id}/start-readiness	Dast start readiness	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.DASTOut
POST	/api/dast-requests/{req_id}/start-scan	Dast start scan	SECURITY_ANALYST	schemas.DASTScanStartOut
POST	/api/dast-requests/{req_id}/submit	Submit dast	Authenticated	schemas.DASTOut
POST	/api/dast-requests/{req_id}/validate-findings	Dast validate findings	SECURITY_ANALYST	schemas.DASTOut
GET	/api/sast-requests	List sast	Authenticated	pagination.Page[schemas.SASTListOut]
POST	/api/sast-requests	Create sast	Authenticated	schemas.SASTOut
GET	/api/sast-requests/{req_id}	Get sast	Authenticated	schemas.SASTOut
PUT	/api/sast-requests/{req_id}	Update sast	Authenticated	schemas.SASTOut
POST	/api/sast-requests/{req_id}/assign-security-analyst	Sast assign security analyst	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA, SECURITY_ANALYST	schemas.SASTOut
POST	/api/sast-requests/{req_id}/assign-to-requester	Sast assign to requester	SECURITY_ANALYST	schemas.SASTOut
GET	/api/sast-requests/{req_id}/checklist	Get sast checklist	Authenticated	List[schemas.ChecklistItemOut]
GET	/api/sast-requests/{req_id}/checklist/documents	Batched counterpart to list_sast_checklist_documents below -- see ChecklistItemDocumentOut for why this exists.	Authenticated	List[schemas.ChecklistItemDocumentOut]
PUT	/api/sast-requests/{req_id}/checklist/{item_id}	Update sast checklist item	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.ChecklistItemOut

Method	Path	Operation	Declared access	Response
GET	/api/sast-requests/{req_id}/checklist/{item_id}/documents	List sast checklist documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/sast-requests/{req_id}/checklist/{item_id}/documents	Upload sast checklist documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/sast-requests/{req_id}/checklist/{item_id}/documents/{doc_id}	Delete sast checklist document	Authenticated	—
GET	/api/sast-requests/{req_id}/checklist/{item_id}/documents/{doc_id}/download	Download sast checklist document	Authenticated	—
POST	/api/sast-requests/{req_id}/close	Sast close request	SECURITY_ANALYST	schemas.SASTOut
POST	/api/sast-requests/{req_id}/department-head-decision	Sast department head decision	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	schemas.SASTOut
GET	/api/sast-requests/{req_id}/documents	List sast documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/sast-requests/{req_id}/documents	Upload sast documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/sast-requests/{req_id}/documents/{doc_id}	Delete sast document	Authenticated	—
GET	/api/sast-requests/{req_id}/documents/{doc_id}/download	Download sast document	Authenticated	—
GET	/api/sast-requests/{req_id}/export	Every field on this SAST request (including the ones delegated from its parent QA Request gateway), its repository components, its findings,	Authenticated	—
POST	/api/sast-requests/{req_id}/findings/{finding_id}/resolve	Resolve sast finding	SECURITY_ANALYST	schemas.SASTFindingOut
GET	/api/sast-requests/{req_id}/history	Sast history	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/sast-requests/{req_id}/mark-fixed	Sast mark fixed	Authenticated	schemas.SASTOut
POST	/api/sast-requests/{req_id}/mark-report-ready	Sast mark report ready	SECURITY_ANALYST	schemas.SASTOut
POST	/api/sast-requests/{req_id}/mark-scan-complete	Sast mark scan complete	SECURITY_ANALYST	schemas.SASTOut
POST	/api/sast-requests/{req_id}/readiness-decision	Sast readiness decision	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.SASTOut
POST	/api/sast-requests/{req_id}/rescan	Sast rescan	SECURITY_ANALYST	schemas.SASTScanStartOut
POST	/api/sast-requests/{req_id}/resubmit	Resubmit sast	Authenticated	schemas.SASTOut
GET	/api/sast-requests/{req_id}/scan-results	Sast scan results	Authenticated	List[schemas.SecurityScanResultOut]
GET	/api/sast-requests/{req_id}/scan-summary	Sast scan summary	Authenticated	schemas.SecurityScanSummaryOut
POST	/api/sast-requests/{req_id}/sm-decision	Sast sm decision	SM	schemas.SASTOut
POST	/api/sast-requests/{req_id}/start-configuration	Sast start configuration	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.SASTOut
POST	/api/sast-requests/{req_id}/start-readiness	Sast start readiness	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.SASTOut
POST	/api/sast-requests/{req_id}/start-scan	Sast start scan	SECURITY_ANALYST	schemas.SASTScanStartOut
POST	/api/sast-requests/{req_id}/submit	Submit sast	Authenticated	schemas.SASTOut
POST	/api/sast-requests/{req_id}/validate-findings	Sast validate findings	SECURITY_ANALYST	schemas.SASTOut

signoff router

15 method/path registrations. Source: backend/app/routers/signoff.py

Method	Path	Operation	Declared access	Response
GET	/api/signoffs	List signoffs	Authenticated	List[schemas.SignOffOut]
POST	/api/signoffs	Raised by the QA Engineer who executed testing -- or, 2026-08 (reported directly: ""Request Sign Off" button is not enable[d] for QA lead ..	QA_ENGINEER, QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.SignOffOut
GET	/api/signoffs/{signoff_id}	Get signoff	Authenticated	schemas.SignOffOut
PUT	/api/signoffs/{signoff_id}	Two different actors, two different windows: the QA requester can edit while the certificate is DRAFT or sitting back with them after a QA L	Authenticated	schemas.SignOffOut
POST	/api/signoffs/{signoff_id}/department-head-coe-decision [hidden alias]	Final COE - Quality Assurance approval by Executive (the QA Executive Group); approval issues the certificate.	CHIEF_MANAGER_QA, AGM_QA	schemas.SignOffOut
GET	/api/signoffs/{signoff_id}/documents	List signoff documents	Authenticated	List[schemas.RequestDocumentOut]
POST	/api/signoffs/{signoff_id}/documents	Upload signoff documents	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/signoffs/{signoff_id}/documents/{doc_id}	Delete signoff document	Authenticated	—
GET	/api/signoffs/{signoff_id}/documents/{doc_id}/download	Download signoff document	Authenticated	—
POST	/api/signoffs/{signoff_id}/executive-coe-decision	Final COE - Quality Assurance approval by Executive (the QA Executive Group); approval issues the certificate.	CHIEF_MANAGER_QA, AGM_QA	schemas.SignOffOut
GET	/api/signoffs/{signoff_id}/export	Every field on this QA Clearance Certificate, plus who requested, reviewed and approved it and when, as one downloadable PDF -- the offline/	Authenticated	—
GET	/api/signoffs/{signoff_id}/history	Signoff history	Authenticated	List[schemas.ApprovalActionOut]
POST	/api/signoffs/{signoff_id}/qa-lead-decision	QA Lead approval checkpoint before Executive final approval.	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.SignOffOut
POST	/api/signoffs/{signoff_id}/resubmit	Re-submits a certificate returned by QA Lead or Executive -- or reopens one rejected by QA Lead (SM_REJECTED; see SIGNOFF_STATUS_LABELS -- t	Authenticated	schemas.SignOffOut
POST	/api/signoffs/{signoff_id}/submit	Submit signoff	Authenticated	schemas.SignOffOut

suppression router

16 method/path registrations. Source: backend/app/routers/suppression.py

Method	Path	Operation	Declared access	Response
GET	/api/suppressions	List suppressions	Authenticated	List[schemas.SuppressionOut]
POST	/api/suppressions	Requester raises the suppression request with all details -- starts life as a Draft (see models.SuppressionRequest.status default) until exp	Authenticated	schemas.SuppressionOut
GET	/api/suppressions/{sup_id}	Get suppression	Authenticated	schemas.SuppressionOut
PUT	/api/suppressions/{sup_id}	Update suppression	Authenticated	schemas.SuppressionOut

Method	Path	Operation	Declared access	Response
POST	/api/suppressions/{sup_id}/dept-head-decision	Dept head decision	DEPARTMENT_HEAD_CM, DEPARTMENT_HEAD_AGM	schemas.Suppression Out
GET	/api/suppressions/{sup_id}/documents	List suppression documents	Authenticated	List[schemas.Request DocumentOut]
POST	/api/suppressions/{sup_id}/documents	Upload suppression documents	Authenticated	List[schemas.Request DocumentOut]
DELETE	/api/suppressions/{sup_id}/documents/{doc_id}	Delete suppression document	Authenticated	—
GET	/api/suppressions/{sup_id}/documents/{doc_id}/download	Download suppression document	Authenticated	—
GET	/api/suppressions/{sup_id}/export	Every field on this Suppression / False Positive request, every finding it covers, and its full approval/workflow history -- who submitted,	Authenticated	—
GET	/api/suppressions/{sup_id}/history	Suppression history	Authenticated	List[schemas.Approval ActionOut]
POST	/api/suppressions/{sup_id}/relink	Reported directly: "give option to link and delink suppression request from sast request and suppression both." Unlike update_suppression above	Authenticated	schemas.Suppression Out
POST	/api/suppressions/{sup_id}/resubmit	Resubmit suppression	Authenticated	schemas.Suppression Out
POST	/api/suppressions/{sup_id}/security-team-decision	Security Team verifies the suppression is legitimate and accepts (Done -- unlocks the linked SAST/DAST request's mark-report- ready), reject	SECURITY_ANALYST	schemas.Suppression Out
POST	/api/suppressions/{sup_id}/sm-decision	SM assigns the request on to the Department Head (Approve), sends it back to the requester (Return), or rejects it outright.	SM	schemas.Suppression Out
POST	/api/suppressions/{sup_id}/submit	Submit suppression	Authenticated	schemas.Suppression Out

test_execution router

40 method/path registrations. Source: backend/app/routers/test_execution.py

Method	Path	Operation	Declared access	Response
DELETE	/api/test-execution/cycle-folders/{folder_id}	Governance-tier action, same as Test Repository's own delete_folder (require_can_manage_execution_governan ce -- QA Lead Group only; Admin al	*_EXEC_ROLES	—
PATCH	/api/test-execution/cycle-folders/{folder_id}	Rename cycle folder	*_EXEC_ROLES	schemas.TestCycleFol derOut
GET	/api/test-execution/cycle-folders/{folder_id}/access	List cycle folder access	Authenticated	List[schemas.TestCycl eFolderAccessOut]
POST	/api/test-execution/cycle-folders/{folder_id}/access	Grants restricted access to this folder to a department or a specific user -- see models.TestCycleFolder's own docstring for the restrict-no	*_EXEC_ROLES	schemas.TestCycleFol derAccessOut
DELETE	/api/test-execution/cycle-folders/{folder_id}/access/ {grant_id}	Same gate as create_cycle_folder_access above -- see that function's own docstring.	*_EXEC_ROLES	—
DELETE	/api/test-execution/cycles/{cycle_id}	Delete a Test Cycle.	*_EXEC_ROLES	—
GET	/api/test-execution/cycles/{cycle_id}	Get cycle	Authenticated	schemas.TestCycleOut
PATCH	/api/test-execution/cycles/{cycle_id}	Edit cycle metadata and enforce the controlled five-state lifecycle.	*_EXEC_ROLES	schemas.TestCycleOut
GET	/api/test-execution/cycles/{cycle_id}/candidate-test-cases	Cursor-paginated approved testcases eligible for this cycle.	Authenticated	schemas.TestCaseCan didatePage

Method	Path	Operation	Declared access	Response
GET	/api/test-execution/cycles/{cycle_id}/executions	SRS 7.2 pagination rollout -- was previously "every execution slot in the cycle" in one unpaginated call.	Authenticated	pagination.Page[schemas.TestExecutionOut]
POST	/api/test-execution/cycles/{cycle_id}/executions	Adds one or more existing Repository test cases to this cycle, creating a Not-Executed TestExecution row for each -- silently skips any that	* _EXEC_ROLES	List[schemas.TestExecutionOut]
POST	/api/test-execution/cycles/{cycle_id}/executions/bulk-assign	Atomically assign one runner to selected testcase slots in a cycle.	* _EXEC_ROLES	List[schemas.TestExecutionOut]
POST	/api/test-execution/cycles/{cycle_id}/executions/bulk-remove	Remove several testcase slots, their attempts, linked defects and evidence metadata from one cycle as one governed database transaction.	QA_ENGINEER, QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.TestExecutionBulkRemoveResult
POST	/api/test-execution/cycles/{cycle_id}/executions/bulk-result	Record one new attempt on every selected testcase slot.	* _EXEC_ROLES	List[schemas.TestExecutionOut]
GET	/api/test-execution/cycles/{cycle_id}/executions/case-ids	PAG-010 -- deliberately NOT paginated, same reasoning as Test Cases' own `/test-cases/all`.	Authenticated	List[int]
POST	/api/test-execution/cycles/{cycle_id}/executions/from-selection	Add explicit IDs or every testcase matching the server-side filter.	* _EXEC_ROLES	schemas.TestExecutionAddResult
GET	/api/test-execution/cycles/{cycle_id}/executions/summary	The progress bar, assignment stat, "My queue" count, the All/Mine/ Unassigned tab bar, and the per-result-status tab bar all used to be comp	Authenticated	schemas.TestExecutionSummaryOut
GET	/api/test-execution/cycles/{cycle_id}/export-xlsx	Export one complete test lifecycle, including every retained run.	Authenticated	—
POST	/api/test-execution/cycles/{cycle_id}/export-xlsx/jobs	Queue test cycle export	Authenticated	—
DELETE	/api/test-execution/cycles/{cycle_id}/request-link	Unlink cycle request	* _EXEC_ROLES	schemas.TestCycleOut
GET	/api/test-execution/executions/blocked-or-failed	2026-08 -- reported directly: on Defect Management's page load, "if there are 30 project[s] then 30 api call[s] ...	Authenticated	List[schemas.DefectLinkableExecutionOut]
DELETE	/api/test-execution/executions/{execution_id}	Removes a test case from a cycle entirely (not the same as recording a result) -- e.g.	QA_ENGINEER, QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	—
GET	/api/test-execution/executions/{execution_id}	PAG-006-adjacent -- didn't exist before this pagination pass, because the list endpoint used to hand back every execution in the cycle at on	Authenticated	schemas.TestExecutionOut
PATCH	/api/test-execution/executions/{execution_id}	Records a new execution attempt (see models.TestExecutionRun) -- reported directly, this used to overwrite the row's own result in place, si	* _EXEC_ROLES	schemas.TestExecutionOut
PATCH	/api/test-execution/executions/{execution_id}/assign	COE - Quality Assurance runner management for one testcase slot in a cycle.	* _EXEC_ROLES	schemas.TestExecutionOut
GET	/api/test-execution/executions/{execution_id}/result-images	List result images	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/test-execution/executions/{execution_id}/result-images/{document_id}	Delete result image	* _EXEC_ROLES	—
GET	/api/test-execution/executions/{execution_id}/result-images/{document_id}/download	Download result image	Authenticated	—
POST	/api/test-execution/executions/{execution_id}/rich-result	Save formatted Actual Result text and pasted/uploaded screenshots as a new execution attempt (see models.TestExecutionRun) -- reported direc	* _EXEC_ROLES	schemas.TestExecutionOut
GET	/api/test-execution/executions/{execution_id}/runs	Full attempt-by-attempt history for this test case's slot in this cycle, oldest first -- see models.TestExecutionRun.	Authenticated	List[schemas.TestExecutionRunOut]
POST	/api/test-execution/executions/{execution_id}/runs/{run_id}/defects	Add run defect	* _EXEC_ROLES	schemas.TestRunDefectOut

Method	Path	Operation	Declared access	Response
DELETE	/api/test-execution/executions/{execution_id}/runs/{run_id}/defects/{defect_id}	Remove run defect	*_EXEC_ROLES	—
GET	/api/test-execution/executions/{execution_id}/runs/{run_id}/images	List run images	Authenticated	List[schemas.RequestDocumentOut]
DELETE	/api/test-execution/executions/{execution_id}/runs/{run_id}/images/{document_id}	Delete run image	*_EXEC_ROLES	—
GET	/api/test-execution/executions/{execution_id}/runs/{run_id}/images/{document_id}/download	Download run image	Authenticated	—
POST	/api/test-execution/executions/{execution_id}/upgrade-version	SRS CYC-006 "An authorized user may upgrade an unexecuted cycle item to a newer approved version after reviewing a change summary."	*_EXEC_ROLES	schemas.TestExecutionOut
GET	/api/test-execution/projects/{project_id}/cycle-folders	A restricted folder this user has no access to is omitted entirely, not just its cycles -- a restricted folder's very existence is part of w	Authenticated	schemas.TestCycleFolderListOut
POST	/api/test-execution/projects/{project_id}/cycle-folders	Create cycle folder	*_EXEC_ROLES	schemas.TestCycleFolderOut
GET	/api/test-execution/projects/{project_id}/cycles	List cycles	Authenticated	pagination.Page[schemas.TestCycleOut]
POST	/api/test-execution/projects/{project_id}/cycles	SRS CYC-001 -- name, type, dates, owner, environment, build and an optional request link are all captured at creation.	*_EXEC_ROLES	schemas.TestCycleOut

test_projects router

16 method/path registrations. Source: backend/app/routers/test_projects.py

Method	Path	Operation	Declared access	Response
GET	/api/test-projects	List test projects	Authenticated	pagination.Page[schemas.TestProjectOut]
POST	/api/test-projects	Create test project	*_MANAGE_ROLES	schemas.TestProjectOut
GET	/api/test-projects/eligible-users	Reported directly: "everywhere in test management whenever asking for users/members just show only users from COE - Quality Assurance, and m	Authenticated	List[schemas.UserOut]
GET	/api/test-projects/{project_id}	Get test project	Authenticated	schemas.TestProjectOut
PATCH	/api/test-projects/{project_id}	Reported directly: "Once Project is created, give option to edit details" (name/application link/department/description all editable by eith	*_MANAGE_ROLES	schemas.TestProjectOut
POST	/api/test-projects/{project_id}/activation-review	QA Lead (or Admin) resolves a pending activate/deactivate request from a QA Engineer -- Approve applies the requested value to is_active, Re	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.TestProjectOut
POST	/api/test-projects/{project_id}/archive	PRJ-003 -- a more deliberate retirement than a plain deactivation, reserved for QA Lead/Admin (no QA-Engineer-request path, unlike is_active	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.TestProjectOut
GET	/api/test-projects/{project_id}/members	2026-08 "Simplified Test Management Review and Approval" requirement (TM-PROJ-002 "no project member-management UI") -- project membership i	Authenticated	List[schemas.TestProjectMemberOut]
POST	/api/test-projects/{project_id}/members	Retained temporarily for older clients; project membership management is disabled -- see list_project_members' docstring above.	*_MANAGE_ROLES	schemas.TestProjectMemberOut

Method	Path	Operation	Declared access	Response
DELETE	/api/test-projects/{project_id}/members/{member_id}	Retained temporarily for older clients; project membership management is disabled -- see list_project_members' docstring above.	* _MANAGE_ROLES	—
PATCH	/api/test-projects/{project_id}/members/{member_id}	Retained temporarily for older clients; project membership management is disabled -- see list_project_members' docstring above.	* _MANAGE_ROLES	schemas.TestProjectMemberOut
GET	/api/test-projects/{project_id}/my-access	SRS PRJ-005/GOV-001 -- what the signed-in user is actually allowed to do on THIS project, combining their project-level role (if they're a m	Authenticated	schemas.TestProjectMyAccessOut
POST	/api/test-projects/{project_id}/unarchive	Lifts is_archived only -- the project comes back as Inactive, not Active, so reactivating it for new work is still its own deliberate decision	QA_LEAD, CHIEF_MANAGER_QA, AGM_QA	schemas.TestProjectOut
GET	/api/test-projects/{project_id}/view-access	2026-08 "view-only access to department/user" CR.	Authenticated	List[schemas.TestProjectViewGrantOut]
POST	/api/test-projects/{project_id}/view-access	Grants read-only visibility into this ONE project (its Test Execution/Repository/Reports, plus its linked Defects) to a department or a spec	Authenticated	schemas.TestProjectViewGrantOut
DELETE	/api/test-projects/{project_id}/view-access/{grant_id}	Delete project view grant	Authenticated	—

test_reports router

5 method/path registrations. Source: backend/app/routers/test_reports.py

Method	Path	Operation	Declared access	Response
GET	/api/test-reports/cycles/{cycle_id}/progress	SRS "Cycle progress": Not Executed/Pass/Fail/Blocked/NA/Retest Passed counts, assignment coverage, and completion percentage -- reads TestEx	Authenticated	schemas.CycleProgressOut
GET	/api/test-reports/portfolio	Cross-project rollout -- the one report in this set that is deliberately NOT scoped to a single project, department-scoped instead (same conv	Authenticated	schemas.ProjectPortfolioOut
GET	/api/test-reports/projects/{project_id}/defect-quality	Defects linked in this project's cycles, grouped by module and by defect status; retest success rate approximates "leakage" as the share of	Authenticated	schemas.DefectQualityOut
GET	/api/test-reports/projects/{project_id}/repository-health	Cases by status/module/priority/type/owner, plus average age and a "never executed" count as the closest honest proxy for "unused duration"	Authenticated	schemas.RepositoryHealthOut
GET	/api/test-reports/projects/{project_id}/version-impact	Cycles carrying stale-pinned items -- SRS TestExecution.is_pinned_stale already computes "the testcase's approved version has moved on since	Authenticated	schemas.VersionImpactOut

test_repository router

45 method/path registrations. Source: backend/app/routers/test_repository.py

Method	Path	Operation	Declared access	Response
DELETE	/api/test-repository/folders/{folder_id}	Folder deletion is a governance action (GOV-002 maker-checker tier).	* _AUTHOR_ROLES	—

Method	Path	Operation	Declared access	Response
PATCH	/api/test-repository/folders/{folder_id}	Reported directly: "Once folder is created, folder details should be editable." A folder's only real "detail" beyond its position in the tree	*_AUTHOR_ROLES	schemas.TestFolderOutput
POST	/api/test-repository/folders/{folder_id}/copy	Duplicates a folder and its entire subtree -- child folders and every test case inside, at any depth -- into a new, independent copy.	*_AUTHOR_ROLES	schemas.TestFolderOutput
POST	/api/test-repository/folders/{folder_id}/move	Re-parents a folder (and everything already nested beneath it, which moves along with it automatically since children only reference their parent)	*_AUTHOR_ROLES	schemas.TestFolderOutput
GET	/api/test-repository/import-template	Serves the canonical "Test Cases - CR-XX - Template" xlsx that import_test_cases() below expects, so users import test cases using this template	Authenticated	—
GET	/api/test-repository/projects/{project_id}/export-xlsx	Export the complete project repository and its QA review history.	Authenticated	—
POST	/api/test-repository/projects/{project_id}/export-xlsx/jobs	Queue test repository export	Authenticated	—
GET	/api/test-repository/projects/{project_id}/folders	List folders	Authenticated	List[schemas.TestFolderOutput]
POST	/api/test-repository/projects/{project_id}/folders	Create folder	*_AUTHOR_ROLES	schemas.TestFolderOutput
POST	/api/test-repository/projects/{project_id}/import-xlsx	Parses the attached "Test Cases - CR-XX - Template" xlsx format: one row per test step, with the test case's own descriptive fields (Epic ID)	*_AUTHOR_ROLES	schemas.TestCaseImportResult
POST	/api/test-repository/projects/{project_id}/import-xlsx/jobs	Upload immediately, then parse and insert outside the HTTP request.	*_AUTHOR_ROLES	—
GET	/api/test-repository/projects/{project_id}/test-cases	SRS 7.2 pagination rollout -- was previously "every test case in the project", with the Repository UI grouping the complete result into its	Authenticated	pagination.Page[schemas.TestCaseListOutput]
POST	/api/test-repository/projects/{project_id}/test-cases	Create test case	*_AUTHOR_ROLES	schemas.TestCaseOutput
GET	/api/test-repository/projects/{project_id}/test-cases/all	PAG-010 -- deliberately NOT paginated, unlike list_test_cases above.	Authenticated	List[schemas.TestCaseListOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-approve	QA-Lead-tier bulk FINAL decision (2026-08 Approval Workflow refactor) -- approves several "Review Completed" draft versions in one atomic decision	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-archive	2026-08 -- "Final-Approved Test Case Deletion and Archive Requirement": bulk counterpart to archive_test_case, alongside bulk_delete_test_case	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-delete	2026-08 "Recycle Bin" requirement -- see delete_test_case's own docstring for the full background.	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-purge	Bulk counterpart to purge_test_case above -- also QA Lead Group only, also permanent.	*_AUTHOR_ROLES	—
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-recommend	Stage-1-tier bulk equivalent of bulk_approve_test_cases above -- recommends several pending draft versions for QA Lead final approval in one	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-reject	NEW-workflow-only bulk equivalent of review_test_case's single-case REJECT decision (terminal) -- see bulk_return_test_cases immediately above	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-restore-from-archive	2026-08 -- bulk counterpart to restore_test_case, alongside bulk_archive_test_cases -- lets an authorized user restore several Archived test cases	*_AUTHOR_ROLES	List[schemas.TestCaseOutput]

Method	Path	Operation	Declared access	Response
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-restore-from-recycle-bin	Bulk counterpart to restore_test_case_from_recycle_bin above.	* _AUTHOR_ROLES	List[schemas.TestCaseOut]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-return	NEW-workflow-only bulk equivalent of review_test_case's single-case RETURN decision -- reported directly, alongside the equivalent bulk Reje	* _AUTHOR_ROLES	List[schemas.TestCaseOut]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-submit	REV-001, bulk form.	* _AUTHOR_ROLES	List[schemas.TestCaseOut]
POST	/api/test-repository/projects/{project_id}/test-cases/bulk-update	Bulk classification changes (priority/test type/module/tags/folder) apply to each selected case's CURRENT draft version if one is in progres	* _AUTHOR_ROLES	List[schemas.TestCaseOut]
GET	/api/test-repository/projects/{project_id}/test-cases/recycle-bin	2026-08 "Recycle Bin" requirement -- everything currently soft- deleted for this project (delete_test_case/bulk_delete_test_cases), awaiting	Authenticated	pagination.Page[schemas.TestCaseListOut]
GET	/api/test-repository/projects/{project_id}/test-cases/summary	The folder tree's per-folder/unfiled counts, the tag filter dropdown's options, and the "Test cases / Approved / Pending review / Critical"	Authenticated	schemas.TestCaseSummaryOut
GET	/api/test-repository/test-cases/by-key/{test_case_key}	Resolve a globally unique test-case business ID for Global Search.	Authenticated	schemas.TestCaseOut
DELETE	/api/test-repository/test-cases/{case_id}	A testcase that has ever been approved OR rejected is governed history -- Archive it instead (TC-006 preserves versions/cycle membership/exe	* _AUTHOR_ROLES	schemas.TestCaseOut
GET	/api/test-repository/test-cases/{case_id}	Get test case	Authenticated	schemas.TestCaseOut
PATCH	/api/test-repository/test-cases/{case_id}	SRS VER-003 "Editing an approved testcase shall create a new Draft version without modifying the approved baseline." If a draft revision is	* _AUTHOR_ROLES	schemas.TestCaseOut
PATCH	/api/test-repository/test-cases/{case_id}/approvers	Retained temporarily for older clients; individual routing is disabled.	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/archive	TC-006 -- archives the approved baseline: preserves all versions, cycle membership, and execution history while preventing new cycle selecti	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/checkin	Releases a checkout.	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/checkout	Reported directly: "check in checkout option should be available for testcases, otherwise multiple people can edit at once, if checkout, the	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/checkout-override	TC-004 "QA Lead and Administrator override shall require a reason and audit event" -- forcibly takes the checkout away from whoever currentl	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/clone	TC-005 -- creates a NEW testcase identity at version 1.0 Draft, recording the source testcase/version it was cloned from.	* _AUTHOR_ROLES	schemas.TestCaseOut
DELETE	/api/test-repository/test-cases/{case_id}/purge	2026-08 "Recycle Bin" requirement -- "only QA lead can clear from recycle bin." The only remaining code path that issues a real, irreversibl	* _AUTHOR_ROLES	—
POST	/api/test-repository/test-cases/{case_id}/restore	Appendix A: Archived -> "optionally restore through approval" -> Approved.	* _AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/restore-from-recycle-bin	2026-08 "Recycle Bin" requirement -- undoes a delete_test_case/ bulk_delete_test_cases soft-delete, same Author-tier access that could delet	* _AUTHOR_ROLES	schemas.TestCaseOut

Method	Path	Operation	Declared access	Response
POST	/api/test-repository/test-cases/{case_id}/review	One endpoint, four possible stages, gated and validated differently depending on which stage the target draft is currently sitting in (see s	*_AUTHOR_ROLES	schemas.TestCaseOut
POST	/api/test-repository/test-cases/{case_id}/submit	REV-001 -- submits the current Draft/Rework-Required version for QA Lead review.	*_AUTHOR_ROLES	schemas.TestCaseOut
GET	/api/test-repository/test-cases/{case_id}/versions	List test case versions	Authenticated	List[schemas.TestCaseVersionSummary]
GET	/api/test-repository/test-cases/{case_id}/versions-compare	SRS VER-005 -- field-level and step-level differences between any two versions of the same testcase.	Authenticated	schemas.TestCaseVersionCompareOut
GET	/api/test-repository/test-cases/{case_id}/versions/{version_id}	Get test case version	Authenticated	schemas.TestCaseVersionOut

Appendix E. Generated inventory provenance

Generated on 23 August 2026 from:

- backend/app/models.py and SQLAlchemy Base.metadata for table, column, key, index and constraint metadata.
- backend/app/routers/*.py FastAPI decorators for method/path registrations, response models and declared dependencies.
- backend/app/main.py, database.py, cache.py, documents.py, backend/alembic and Docker deployment configuration for runtime and migration boundaries.
- frontend/src/App.tsx and domain modules for client composition and workflow surfaces.

Limitation: this is an implementation reference, not a live database introspection report. A deployment may differ until every reviewed Alembic revision has been applied and verified against the target Oracle schema.